PostgreSQL - Sistema de Gerenciamento de Banco de Dados

Introdução ao PostgreSQL

PostgreSQL Logo

PostgreSQL é um sistema de gerenciamento de banco de dados relacional a objetos (ORDBMS) de código aberto, reconhecido pela confiabilidade, robustez e alto desempenho em ambientes corporativos. Com mais de 35 anos de desenvolvimento contínuo, o PostgreSQL suporta tanto tipos de dados SQL (relacionais) quanto JSON (não relacionais), sendo ideal para aplicações complexas, pipelines de dados, análise avançada e sistemas geográficos através da extensão PostGIS.

Principais Características

Característica	Descrição
Código Aberto (Open Source)	Gratuito e flexível, sem custos de licenciagem, permitindo customizações conforme necessário
Compatibilidade ACID	Garante transações seguras com alta integridade e consistência de dados, essencial para pipelines de ETL
Extensibilidade	Permite definir tipos de dados customizados, criar funções personalizadas e utilizar extensões como PostGIS e TimescaleDB
Versatilidade de Dados	Suporta dados relacionais, JSON, arrays e tipos geoespaciais, ideal para arquiteturas modernas de dados
Comunidade Ativa	Mantido por equipe global com atualizações regulares, garantindo segurança e compatibilidade com novas tecnologias
Funcionalidades Avançadas	Suporta chaves estrangeiras, subconsultas otimizadas, triggers, views materializadas e window functions

Por que Escolher PostgreSQL para Engenharia de Dados

O PostgreSQL é a escolha preferida de engenheiros de dados por sua capacidade de lidar com grandes volumes de dados, suportar transformações complexas através de SQL avançado, integrar-se facilmente com ferramentas de BI e ETL, e oferecer excelente relação custo-benefício em ambientes corporativos.

Estrutura do Banco de Dados - Projeto Apex Motors

O projeto de concessionária **Apex Motors** foi desenvolvido com uma arquitetura relacional robusta, utilizando PostgreSQL como base de dados. A estrutura é composta por sete tabelas principais que se relacionam através de chaves estrangeiras, garantindo integridade referencial e consistência dos dados.

Diagrama de Relacionamento das Tabelas

Estrutura das Tabelas

A estrutura acima apresenta o relacionamento entre as seguintes tabelas:

- **estados**: Armazena informações dos estados brasileiros
- **cidades**: Relaciona cidades aos seus respectivos estados
- **concessionarias**: Armazena dados das concessionárias Apex Motors em diferentes cidades
- **clientes**: Mantém registro de clientes que realizam compras
- **vendedores**: Profissionais de vendas vinculados às concessionárias
- **veiculos**: Catálogo de veículos disponíveis para venda
- **vendas**: Tabela central que relaciona todas as transações de venda

---

Criação das Tabelas

A seguir, apresentamos o script SQL completo para criação de todas as tabelas do sistema:

1. Tabela de Estados

```sql
CREATE TABLE estados (
    id_estados SERIAL PRIMARY KEY,
    estado VARCHAR(100) NOT NULL,
    sigla CHAR(2) NOT NULL,
    data_inclusao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    data_atualicao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

A tabela `estados` armazena informações dos estados brasileiros, incluindo o nome completo e a sigla de dois caracteres. Os campos de data permitem auditoria do sistema.

2. Tabela de Cidades

```sql
CREATE TABLE cidades (
    id_cidades SERIAL PRIMARY KEY,
    cidade VARCHAR(255) NOT NULL,
    id_estados INT NOT NULL,
    data_inclusao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    data_atualicao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_cidades_estados
        FOREIGN KEY (id_estados) REFERENCES estados(id_estados)
);
```

A tabela `cidades` mantém o registro de cidades onde as concessionárias estão localizadas, com referência obrigatória a um estado.

3. Tabela de Concessionárias

```
CREATE TABLE concessionarias (
    id_concessionarias SERIAL PRIMARY KEY,
    concessionaria VARCHAR(255) NOT NULL,
    id_cidades INT NOT NULL,
    data_inclusao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    data_atualicao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_concessionarias_cidades
        FOREIGN KEY (id_cidades) REFERENCES cidades(id_cidades)
);
```

A tabela `concessionarias` armazena as unidades de negócio Apex Motors, cada uma vinculada a uma cidade específica.

4. Tabela de Vendedores

```
CREATE TABLE vendedores (
    id_vendedores SERIAL PRIMARY KEY,
    nome VARCHAR(255) NOT NULL,
    id_concessionarias INT NOT NULL,
    data_inclusao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    data_atualicao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_vendedores_concessionarias
        FOREIGN KEY (id_concessionarias) REFERENCES
concessionarias(id_concessionarias)
);
```

A tabela `vendedores` registra os profissionais de vendas, cada um vinculado a uma concessionária específica.

5. Tabela de Clientes

```
CREATE TABLE clientes (
    id_clientes SERIAL PRIMARY KEY,
    cliente VARCHAR(255) NOT NULL,
    endereco TEXT NOT NULL,
    id_concessionarias INT NOT NULL,
    data_inclusao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    data_atualicao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_clientes_concessionarias
        FOREIGN KEY (id_concessionarias) REFERENCES
concessionarias(id_concessionarias)
);
```

A tabela `clientes` mantém informações dos compradores, incluindo endereço completo e a concessionária onde realizaram a compra.

6. Tabela de Veículos

```
CREATE TABLE veiculos (
    id_veiculos SERIAL PRIMARY KEY,
    nome VARCHAR(255) NOT NULL,
    tipo VARCHAR(100) NOT NULL,
    valor DECIMAL(10,2) NOT NULL,
    data_inclusao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    data_atualicao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

A tabela `veiculos` armazena o catálogo de modelos disponíveis, com informações de tipo e valor de venda.

7. Tabela de Vendas

```sql
CREATE TABLE vendas (
    id_vendas SERIAL PRIMARY KEY,
    id_veiculos INT NOT NULL,
    id_concessionarias INT NOT NULL,
    id_vendedores INT NOT NULL,
    id_clientes INT NOT NULL,
    valor_pago DECIMAL(10,2) NOT NULL,
    data_venda TIMESTAMP NOT NULL,
    data_inclusao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    data_atualicao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_vendas_veiculos FOREIGN KEY (id_veiculos) REFERENCES
veiculos(id_veiculos),
    CONSTRAINT fk_vendas_concessionarias FOREIGN KEY (id_concessionarias)
REFERENCES concessionarias(id_concessionarias),
    CONSTRAINT fk_vendas_vendedores FOREIGN KEY (id_vendedores) REFERENCES
vendedores(id_vendedores),
    CONSTRAINT fk_vendas_clientes FOREIGN KEY (id_clientes) REFERENCES
clientes(id_clientes)
);
```

A tabela `vendas` é a tabela central do sistema, relacionando todas as entidades envolvidas em uma transação de venda. Cada venda registra o veículo, a concessionária, o vendedor, o cliente e o valor pago.

Consultas SQL - Análise de Dados

O sistema foi desenvolvido com um conjunto robusto de consultas SQL que permitem análise completa das operações de venda. Abaixo apresentamos as principais consultas implementadas:

Consulta 1: Seleção Simples

Objetivo: Listar todos os veículos do tipo “SUV Urbano/Aventura” com valor inferior a R$ 200.000,00

```sql
SELECT nome, tipo, valor
FROM veiculos
WHERE tipo = 'SUV Urbano/Aventura' AND valor < 200000.00;
```

Esta consulta demonstra o uso de filtros básicos (WHERE) com múltiplas condições (AND), permitindo identificar veículos específicos dentro de critérios de preço. O resultado retorna veículos como Apex Stratos com valor de R$ 159.000,00.

Consulta 2: Junção Simples

Objetivo: Exibir o nome dos clientes e o nome das concessionárias onde realizaram suas compras

```sql
SELECT c.cliente, con.concessionaria
FROM clientes c
JOIN concessionarias con ON c.id_concessionarias = con.id_concessionarias
LIMIT (10);
```

Esta consulta utiliza JOIN para relacionar dados de duas tabelas, permitindo visualizar quais clientes compraram em quais concessionárias. O LIMIT restringe o resultado a 10 registros, mostrando clientes como Mateus da Nascimento na Apex Motors Rio de Janeiro.

Consulta 3: Contagem e Agrupamento

Objetivo: Contar quantos vendedores existem em cada concessionária

```sql
SELECT id_concessionarias, COUNT(*) AS numero_vendedores
FROM vendedores
GROUP BY id_concessionarias
LIMIT (10);
```

Esta consulta utiliza agregação (COUNT) com GROUP BY para agrupar dados por concessionária, fornecendo insights sobre a distribuição de vendedores. Os resultados

mostram que a concessionária 29 possui 2 vendedores, enquanto outras possuem entre 1 e 4.

Consulta 4: Subconsulta

Objetivo: Encontrar os veículos mais caros vendidos em cada tipo de veículo

```
SELECT tipo, MAX(valor) AS valor_maximo
FROM veiculos
GROUP BY tipo;
```

Esta consulta identifica o valor máximo de cada tipo de veículo, útil para análise de precificação e posicionamento de mercado. Os resultados mostram que o Supercar Esportivo é o mais caro (R$459.000,00), $seguidopelaSedãExecutivo(R$ 179.000,00).

Consulta 5: Junção Múltipla

Objetivo: Listar o nome do cliente, o veículo comprado e o valor pago, para todas as vendas

```
SELECT cl.cliente, v.nome AS veiculo, vd.valor_pago
FROM vendas vd
JOIN clientes cl ON vd.id_clientes = cl.id_clientes
JOIN veiculos v ON vd.id_veiculos = v.id_veiculos
LIMIT (10);
```

Esta consulta demonstra o uso de múltiplos JOINs para relacionar três tabelas, fornecendo uma visão completa de cada transação de venda. Exemplos incluem Felipe Gustavo Freitas comprando um Apex Ion por R$ 492.057,22.

Consulta 6: Filtro com Agregação

Objetivo: Identificar as concessionárias que venderam mais de 5 veículos

```sql
SELECT id_concessionarias, COUNT(*) AS total_vendas
FROM vendas
GROUP BY id_concessionarias
HAVING COUNT(*) > 5
LIMIT (10);
```

Esta consulta utiliza HAVING para filtrar grupos que atendem a critérios específicos, identificando as concessionárias com melhor desempenho de vendas. A concessionária 29 lidera com 173 vendas, seguida pela concessionária 4 com 206 vendas.

Consulta 7: Ordenação e Limite

Objetivo: Listar os três veículos mais caros disponíveis

```sql
SELECT nome, valor
FROM veiculos
ORDER BY valor DESC
LIMIT 3;
```

Esta consulta utiliza ORDER BY para ordenação descendente e LIMIT para retornar apenas os registros de maior interesse. Os três veículos mais caros são: Apex Ion (R $499.000,00)$, $ApexVortex(R$ 459.000,00) e Apex Titan (R$ 269.000,00).

Consulta 8: Filtro com Data

Objetivo: Selecionar todos os veículos adicionados nos últimos 4 anos

```sql
SELECT nome, data_inclusao
FROM veiculos
WHERE data_inclusao > CURRENT_TIMESTAMP - INTERVAL '4 year';
```

Esta consulta demonstra o uso de funções de data e intervalo, permitindo análises temporais sobre os dados do sistema. Todos os veículos do catálogo foram incluídos

em 15 de janeiro de 2023, demonstrando uma base de dados consistente.

Consulta 9: Junção Externa

Objetivo: Listar as cidades e qualquer concessionária nelas, se houver

```
SELECT c.cidade, con.concessionaria
FROM cidades c
LEFT JOIN concessionarias con ON c.id_cidades = con.id_cidades
LIMIT(10);
```

Esta consulta utiliza LEFT JOIN para manter todos os registros da tabela esquerda (cidades), mesmo que não haja concessionárias associadas. Exemplos incluem Rio Branco com Apex Motors Rio Branco, Maceió com Apex Motors Maceió.

Consulta 10: Múltiplas Condições

Objetivo: Encontrar clientes que compraram veículos 'SUV Premium Híbrida' ou veículos com valor acima de R$ 200.000,00

```
SELECT cl.cliente, v.nome AS veiculo, v.valor
FROM vendas vd
JOIN veiculos v ON vd.id_veiculos = v.id_veiculos
JOIN clientes cl ON vd.id_clientes = cl.id_clientes
WHERE v.tipo = 'SUV Premium Híbrida' OR v.valor > 200000.00
LIMIT 10;
```

Esta consulta combina múltiplas JOINs com condições complexas (OR), permitindo análises sofisticadas sobre o comportamento de compra dos clientes. Exemplos incluem Felipe Gustavo Freitas com Apex Ion (R $499.000,00)eBeatrizRicardoPereiracomApexTitan(R$ 269.000,00).

Conclusão

O PostgreSQL demonstra ser uma solução robusta e versátil para o gerenciamento do banco de dados da Apex Motors. A estrutura relacional implementada garante integridade referencial, enquanto as consultas SQL avançadas permitem análises profundas sobre operações de venda, desempenho de concessionárias e comportamento de clientes. A combinação de funcionalidades ACID, extensibilidade e comunidade ativa torna o PostgreSQL a escolha ideal para este projeto de engenharia de dados.

Render - Plataforma de Hospedagem do Banco de Dados

Introdução ao Render

Render é uma plataforma moderna de hospedagem em nuvem que oferece soluções simples e eficientes para deploy de aplicações e bancos de dados. Diferentemente de plataformas mais complexas como AWS ou Google Cloud, o Render foi projetado com foco em facilidade de uso, mantendo a robustez e confiabilidade necessárias para ambientes corporativos.

Por que Escolher Render para Hospedagem de Banco de Dados

O Render oferece várias vantagens para projetos que necessitam de hospedagem confiável:

- **Simplicidade**: Interface intuitiva e processo de deploy automatizado
- **Custo-benefício**: Plano gratuito generoso com opções de upgrade acessíveis
- **Integração com Git**: Deploy automático ao fazer push no repositório
- **Segurança**: SSL/TLS automático, backups regulares e isolamento de rede
- **Escalabilidade**: Fácil upgrade de recursos conforme a aplicação cresce
- **Suporte a PostgreSQL**: Instâncias PostgreSQL gerenciadas com alta disponibilidade

---

Configuração da Instância PostgreSQL no Render

Tipo de Instância

A configuração inicial do projeto Apex Motors utiliza o plano **Starter** do Render, que oferece:

Recurso	Quantidade
RAM	512 MB
CPU	0.5 CPU
Storage	10 GB

O plano Starter oferece um excelente equilíbrio entre custo e desempenho, sendo ideal para aplicações em desenvolvimento e produção com volume moderado de dados. Oferece recursos suficientes para suportar múltiplas conexões simultâneas e maior volume de dados comparado ao plano Free.

Alta Disponibilidade (High Availability)

A configuração de Alta Disponibilidade no Render permite que o banco de dados execute uma instância standby que automaticamente assume o controle em caso de falha na instância primária. Esta funcionalidade está **desabilitada** no plano Free, sendo disponível apenas para instâncias Pro e superiores.

A alta disponibilidade é essencial para ambientes críticos onde qualquer downtime representa perda de receita ou dados. Para o projeto Apex Motors em desenvolvimento, a instância padrão do plano Starter é suficiente, mas a funcionalidade pode ser habilitada no futuro se necessário.

Informações Gerais da Instância

Detalhes da Instância

Informações Gerais

A instância PostgreSQL do projeto Apex Motors foi criada com as seguintes características:

Configuração	Valor
Nome do Banco	apex-db
Status	Available (Disponível)
Versão PostgreSQL	18
Região	Oregon (US West)
Data de Criação	Aproximadamente um mês atrás
Armazenamento Utilizado	Aproximadamente 696 MB de 10 GB
Replicação de Leitura	Não configurada
Auto-scaling de Armazenamento	Desabilitado

A escolha da região **Oregon (US West)** oferece boa latência para usuários na América do Norte e é uma região estável do Render com excelente disponibilidade.

Instância PostgreSQL

Tipo de Instância

A instância utiliza a configuração padrão do Render para PostgreSQL, com as seguintes especificações:

- **Tipo**: Starter (512 MB RAM, 0.5 CPU, 10 GB Storage)
- **Plano**: Gratuito com limite de 90 dias de inatividade
- **Backup Automático**: Habilitado
- **SSL/TLS**: Automático

Configuração de Conexão

Detalhes de Conexão

Configurações de Conexão

Para conectar à instância PostgreSQL do Render, as seguintes informações são necessárias:

Parâmetro	Valor
Hostname	dpg-d5a8u3je5dus73avg4cd-s
Port	5432
Database	apex_db_5nni
Username	apex_db_5nni_user
Password	[Armazenado com segurança]
Internal Database URL	[URL privada para comunicação interna]
External Database URL	[URL pública para acesso externo]
PSQL Command	[Comando pré-configurado para conexão]

Explicação dos Parâmetros

Hostname: O endereço interno da instância PostgreSQL no Render. Este é um identificador único que aponta para o servidor de banco de dados.

Port: A porta padrão do PostgreSQL é 5432. O Render utiliza esta porta para todas as instâncias PostgreSQL.

Database: O nome do banco de dados criado, `apex_db_5nni`, específico para o projeto Apex Motors. O sufixo `5nni` é um identificador único gerado pelo Render.

Username: O usuário padrão para conexão é `apex_db_5nni_user`, criado automaticamente pelo Render para este banco de dados.

Internal Database URL: URL de conexão para serviços internos dentro da rede do Render (ex: aplicações também hospedadas no Render).

External Database URL: URL de conexão pública para acessar o banco de dados de fora da rede Render (ex: aplicações locais, servidores externos).

PSQL Command: Comando pré-configurado que pode ser copiado e executado no terminal para conectar diretamente ao banco de dados.

Gerenciamento de Credenciais

Credenciais de Acesso

Gerenciamento de Credenciais

O Render oferece um sistema robusto de gerenciamento de credenciais para segurança máxima:

Aspecto	Descrição
Credencial Padrão	apex_db_5nni_user (criada automaticamente)
Status	Default (Padrão)
Data de Criação	27 dias atrás
Conexões Abertas	1 conexão ativa
Rotação de Senhas	Pode ser feita a qualquer momento

Boas Práticas de Segurança

1. **Senhas Fortes**: O Render gera senhas criptograficamente seguras automaticamente
2. **Rotação Regular**: Recomenda-se rotacionar credenciais a cada 90 dias em produção
3. **Credenciais Múltiplas**: É possível criar múltiplos usuários com diferentes permissões
4. **Armazenamento Seguro**: As credenciais devem ser armazenadas em variáveis de ambiente, nunca hardcoded

Restrições de IP (Networking)

O Render oferece controle granular sobre quais IPs podem acessar o banco de dados:

Validação de IP: Configurado para aceitar conexões de qualquer IP (0.0.0.0), permitindo acesso global. Para ambientes de produção, recomenda-se restringir a IPs específicos.

Regras de Firewall PostgreSQL:

Origem	Descrição
0.0.0.0/0	Qualquer IP pode conectar (Desenvolvimento)

Para produção, seria recomendado alterar para:

- IPs específicos de servidores de aplicação
- Ranges de IP de escritórios corporativos
- IPs estáticos de provedores de hospedagem

Processo de Criação da Instância

Passo a Passo para Criar uma Instância PostgreSQL no Render

1. Acesso ao Dashboard

Acesse render.com e faça login em sua conta. No dashboard, clique em **"New +"** e selecione **"PostgreSQL"**.

2. Configuração Básica

- **Name**: Digite um nome descritivo para sua instância (ex: apex-db)
- **Database**: O nome do banco de dados será gerado automaticamente
- **User**: Um usuário padrão será criado automaticamente
- **Region**: Selecione a região mais próxima de seus usuários
- **PostgreSQL Version**: Escolha a versão mais recente estável

3. Seleção do Plano

O Render oferece os seguintes planos:

Plano	RAM	CPU	Storage	Preço
Free	256 MB	0.1	1 GB	Gratuito
Starter	512 MB	0.5	10 GB	$7/mês
Standard	1 GB	1	20 GB	$15/mês
Pro	4 GB	2	100 GB	$50/mês

Para o projeto Apex Motors, o plano **Starter** foi escolhido por oferecer melhor desempenho e maior capacidade de armazenamento, permitindo suportar a aplicação em desenvolvimento com mais recursos disponíveis.

4. Configurações Avançadas

- **High Availability**: Desabilitado (disponível apenas em planos pagos)
- **Backups**: Habilitados automaticamente
- **SSL/TLS**: Habilitado automaticamente
- **Storage Autoscaling**: Pode ser habilitado para planos pagos

5. Criação e Acesso

Após clicar em **"Create Database"**, o Render provisiona a instância em segundos. As credenciais de conexão ficam imediatamente disponíveis no dashboard.

Conexão com Aplicações

Variáveis de Ambiente

Para conectar sua aplicação ao banco de dados Render, configure as seguintes variáveis de ambiente:

```
DATABASE_URL=postgresql://apex_db_5nni_user:password@dpg-
d5a8u3je5dus73avg4cd-s:5432/apex_db_5nni
DB_HOST=dpg-d5a8u3je5dus73avg4cd-s
DB_PORT=5432
DB_NAME=apex_db_5nni
DB_USER=apex_db_5nni_user
DB_PASSWORD=your_secure_password
```

Exemplo de Conexão com Node.js

```
const { Pool } = require('pg');

const pool = new Pool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  ssl: {
    rejectUnauthorized: false
  }
});

module.exports = pool;
```

Exemplo de Conexão com Python

```python
import psycopg2
from psycopg2 import sql

connection = psycopg2.connect(
    host=os.getenv('DB_HOST'),
    port=os.getenv('DB_PORT'),
    database=os.getenv('DB_NAME'),
    user=os.getenv('DB_USER'),
    password=os.getenv('DB_PASSWORD'),
    sslmode='require'
)

cursor = connection.cursor()
```

Monitoramento e Manutenção

Monitoramento de Recursos

O Render oferece dashboard em tempo real para monitorar:

- **Uso de CPU**: Visualizar picos de uso
- **Uso de Memória**: Acompanhar consumo de RAM
- **Uso de Armazenamento**: Monitorar crescimento do banco de dados
- **Conexões Ativas**: Quantidade de conexões simultâneas
- **Logs**: Acesso a logs de erro e eventos

Backups Automáticos

O Render realiza backups automáticos diários de todas as instâncias PostgreSQL. Os backups são:

- **Retenção**: Mantidos por 7 dias
- **Restauração**: Podem ser restaurados a qualquer momento

- **Ponto de Recuperação**: Permite restaurar para qualquer ponto nos últimos 7 dias

Manutenção Preventiva

O Render realiza atualizações de segurança e patches automaticamente durante janelas de manutenção programadas. Para instâncias Free, estas janelas são mais flexíveis, enquanto instâncias pagas têm janelas de manutenção mais previsíveis.

Escalabilidade e Upgrade

Quando Fazer Upgrade

Considere fazer upgrade do plano quando:

- Armazenamento utilizado exceder 80% do limite
- Conexões simultâneas frequentemente atingem o limite
- CPU ou memória estão consistentemente acima de 80%
- Necessidade de alta disponibilidade (redundância)
- Replicação de leitura para distribuição de carga

Processo de Upgrade

O upgrade no Render é simples e pode ser feito sem downtime:

1. Acesse o dashboard da instância
2. Clique em **"Update"** na seção de tipo de instância
3. Selecione o novo plano
4. Confirme a alteração
5. O Render realiza o upgrade com mínima interrupção

Conclusão

O Render oferece uma solução de hospedagem de banco de dados PostgreSQL que combina simplicidade, segurança e confiabilidade. Para o projeto Apex Motors, a escolha do plano Free permite desenvolvimento e testes sem custos, com a flexibilidade de fazer upgrade conforme a aplicação cresce. A configuração atual, com instância em Oregon, credenciais seguras e acesso global, fornece uma base sólida para o desenvolvimento da aplicação web e mobile do projeto.

A integração do Render com a aplicação é direta e bem documentada, permitindo que desenvolvedores se concentrem na lógica de negócio em vez de gerenciamento de infraestrutura.

Amazon EC2 - Serviço de Computação em Nuvem

Introdução ao Amazon EC2

Amazon EC2 Logo

Amazon Elastic Compute Cloud (Amazon EC2) é um serviço da Web da Amazon Web Services (AWS) que fornece capacidade computacional segura e redimensionável na nuvem, permitindo que usuários aluguem servidores virtuais (chamados de instâncias). Ele elimina a necessidade de hardware físico, oferecendo controle total sobre o sistema operacional e permitindo escalabilidade rápida para atender a demandas de aplicativos.

O EC2 é a base da infraestrutura de computação em nuvem da AWS, sendo utilizado por empresas de todos os tamanhos, desde startups até grandes corporações, para hospedar aplicações web, bancos de dados, servidores de aplicação e muito mais.

Principais Características e Benefícios

Característica	Descrição
Instâncias Virtuais	Servidores virtuais que podem ser criados, configurados e encerrados conforme a necessidade
Capacidade Elástica	Capacidade de aumentar ou diminuir a escala (computação) instantaneamente para lidar com picos de tráfego
Ampla Variedade de Configurações	Opções de CPU, memória, armazenamento e rede para atender a diferentes cargas de trabalho
Pagamento por Uso (Pay-as-you-go)	Os usuários pagam apenas pelo tempo de processamento que as instâncias estão ativas
Segurança	Integra-se com a infraestrutura da AWS para fornecer segurança, incluindo firewall, grupos de segurança e controle de acesso
Controle Total	Acesso root/administrador completo ao sistema operacional e aplicações
Múltiplos Sistemas Operacionais	Suporte para Linux, Windows, macOS e outros sistemas operacionais
Integração com AWS	Integração perfeita com outros serviços AWS como S3, RDS, CloudFront, etc

---

Tipos de Instâncias EC2

O EC2 oferece diversos tipos de instâncias otimizadas para diferentes casos de uso:

Famílias de Instâncias

Família	Caso de Uso	Exemplo
T (Burstable)	Aplicações com carga variável, desenvolvimento, testes	t2.micro, t2.small, t2.large
M (General Purpose)	Aplicações balanceadas, servidores web, aplicações corporativas	m5.large, m6i.xlarge
C (Compute Optimized)	Processamento intensivo, análise de dados, compilação	c5.large, c6i.2xlarge
R (Memory Optimized)	Bancos de dados em memória, cache, processamento de big data	r5.large, r6i.4xlarge
I (Storage Optimized)	Bancos de dados NoSQL, data warehousing, Elasticsearch	i3.large, i4i.2xlarge
G (GPU Instances)	Machine learning, processamento gráfico, renderização	g4dn.xlarge, g5.2xlarge

Instância Utilizada no Projeto Apex Motors

Para o projeto Apex Motors, foi escolhida a instância **t2.large**, que oferece:

Recurso	Quantidade
vCPU	2
Memória (RAM)	8 GB
Armazenamento EBS	Até 160 Mbps
Tipo de Rede	Até 5 Gbps

A instância t2.large é ideal para aplicações web em desenvolvimento e produção com volume moderado de tráfego, oferecendo bom equilíbrio entre custo e desempenho.

Criação da Instância EC2

Passo 1: Acessar o Console AWS

1. Acesse console.aws.amazon.com
2. Faça login com suas credenciais AWS
3. Navegue até o serviço **EC2** através do menu de serviços
4. Clique em **"Instances"** no menu lateral esquerdo
5. Clique no botão **"Launch instances"**

Passo 2: Configuração Básica da Instância

Na tela de launch, configure os seguintes parâmetros:

Nome da Instância

- **Campo**: Name
- **Valor**: Apex_Motors
- **Descrição**: Nome descritivo para identificar a instância no console AWS

Seleção da Imagem (AMI)

A **Amazon Machine Image (AMI)** é um template que contém o sistema operacional e software pré-configurado.

- **Sistema Operacional**: Ubuntu
- **Versão**: 22.04 LTS (recomendado para estabilidade)
- **Tipo**: 64-bit (x86)

Ubuntu é escolhido por ser:

- Gratuito e open-source
- Bem documentado
- Amplamente utilizado em produção
- Excelente suporte da comunidade

Tipo de Instância

- **Tipo**: t2.large
- **vCPU**: 2
- **Memória**: 8 GB RAM
- **Armazenamento**: EBS (Elastic Block Store)

Par de Chaves (Key Pair)

- **Nome**: Key_Pair_Apex_Motors
- **Tipo**: RSA
- **Formato**: .pem (para Linux/Mac) ou .ppk (para PuTTY no Windows)

Importante: A chave privada deve ser armazenada com segurança e nunca compartilhada. Ela é necessária para acessar a instância via SSH.

Passo 3: Configuração de Rede e Segurança

Grupo de Segurança (Security Group)

O grupo de segurança atua como um firewall virtual que controla o tráfego de entrada e saída.

Regras de Entrada (Inbound Rules):

Protocolo	Porta	Origem	Descrição
SSH	22	0.0.0.0/0	Acesso remoto via SSH (qualquer IP)
HTTP	80	0.0.0.0/0	Tráfego web não criptografado
HTTPS	443	0.0.0.0/0	Tráfego web criptografado
TCP Customizado	8080	0.0.0.0/0	Porta para interface gráfica da aplicação

Explicação das Regras:

- **SSH (Porta 22)**: Permite conexão remota via terminal. Essencial para gerenciar a instância.

- **HTTP (Porta 80)**: Permite acesso à aplicação web através de HTTP (não criptografado).
- **HTTPS (Porta 443)**: Permite acesso seguro à aplicação web através de HTTPS (criptografado com SSL/TLS).
- **TCP Customizado (Porta 8080)**: Porta alternativa para acessar a interface gráfica da aplicação Apex Motors.

Regras de Saída (Outbound Rules):

Por padrão, todas as conexões de saída são permitidas (0.0.0.0/0 em todos os protocolos).

Passo 4: Configuração de Armazenamento

- **Tipo de Volume**: General Purpose (gp3)
- **Tamanho**: 30 GB (recomendado para aplicação web com banco de dados)
- **Criptografia**: Habilitada (AWS KMS)
- **Exclusão ao Encerrar**: Habilitada (volume é deletado quando a instância é encerrada)

Passo 5: Lançamento da Instância

1. Revise todas as configurações
2. Clique em **"Launch instance"**
3. Aguarde a instância ser criada (geralmente 1-2 minutos)
4. Anote o **Instance ID** e o **Public IP Address**

Configuração de Portas Adicionais

Após criar a instância, pode ser necessário abrir portas adicionais para acessar serviços específicos.

Abrir Porta 8080 para Interface Gráfica

Passo 1: Acessar o Grupo de Segurança

1. No console EC2, clique em **"Instances"**
2. Selecione a instância **Apex_Motors**
3. Na aba **"Details"**, localize **"Security groups"**
4. Clique no nome do grupo de segurança

Passo 2: Editar Regras de Entrada

1. Na página do grupo de segurança, clique em **"Edit inbound rules"**
2. Clique em **"Add rule"**
3. Configure a nova regra:
   - **Type**: Custom TCP
   - **Port Range**: 8080
   - **Source**: Anywhere IPv4 (0.0.0.0/0)
   - **Description**: Interface Gráfica Apex Motors
4. Clique em **"Save rules"**

A porta 8080 agora está acessível via `http://seu-ip-publico:8080`

Acesso via SSH

Verificar SSH na Máquina Local

Antes de conectar, verifique se SSH está instalado:

No Windows (PowerShell)

```
ssh -V
```

No Linux/Mac (Terminal)

```
ssh -V
```

Se SSH não estiver instalado, instale:

- **Windows**: Use Git Bash ou Windows Subsystem for Linux (WSL)
- **Linux**: `sudo apt-get install openssh-client`
- **Mac**: SSH já vem pré-instalado

Obter as Informações de Conexão

1. No console EC2, clique em **"Instances"**
2. Selecione a instância **Apex_Motors**
3. Clique no botão **"Connect"**
4. Selecione a aba **"SSH client"**
5. Copie o comando SSH fornecido

Conectar à Instância

Passo 1: Navegar até a Chave Privada

```
# Windows PowerShell
cd .\Desktop\apex_motors
dir
```

```
# Linux/Mac Terminal
cd ~/Desktop/apex_motors
ls
```

Passo 2: Ajustar Permissões da Chave (Linux/Mac)

```
chmod 400 key_pair_apex_motors.pem
```

Passo 3: Conectar via SSH

```
ssh -i "key_pair_apex_motors.pem" ubuntu@ec2-100-27-218-149.compute-1.amazonaws.com
```

Nota: Substitua o endereço IP/hostname pelo da sua instância.

Exemplo de Conexão Bem-Sucedida

```
The authenticity of host 'ec2-100-27-218-149.compute-1.amazonaws.com (100.27.218.149)' can't be established.
ECDSA key fingerprint is SHA256:xxxxx...
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-100-27-218-149.compute-1.amazonaws.com,100.27.218.149' (ECDSA) to the list of known_hosts.
Welcome to Ubuntu 22.04.1 LTS (GNU/Linux 5.15.0-1021-aws x86_64)

ubuntu@ip-172-31-0-1:~$
```

Quando você vir o prompt `ubuntu@ip-xxx:~$`, significa que está conectado com sucesso à instância.

Gerenciamento da Instância

Comandos Úteis após Conectar via SSH

Verificar Informações do Sistema

```
# Ver versão do Ubuntu
lsb_release -a

# Ver uso de CPU e memória
top

# Ver espaço em disco
df -h

# Ver informações da rede
ifconfig
```

Instalar Dependências

```
# Atualizar lista de pacotes
sudo apt update

# Atualizar pacotes instalados
sudo apt upgrade

# Instalar Node.js
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt-get install -y nodejs

# Instalar npm (geralmente vem com Node.js)
npm -v

# Instalar Git
sudo apt install git

# Instalar Docker
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

Parar e Iniciar a Instância

Via Console AWS

1. Clique em **"Instances"**
2. Selecione a instância
3. Clique em **"Instance State"**
4. Selecione **"Stop"** (para parar) ou **"Start"** (para iniciar)

Via SSH

```
# Parar a instância (requer permissões AWS)
sudo shutdown -h now

# Reiniciar a instância
sudo reboot
```

Monitorar Performance

O CloudWatch é o serviço de monitoramento da AWS que fornece métricas em tempo real:

1. No console AWS, acesse **"CloudWatch"**
2. Clique em **"Dashboards"**
3. Selecione a instância EC2
4. Visualize gráficos de:
    - CPU Utilization
    - Network In/Out
    - Disk Read/Write
    - Status Checks

Boas Práticas de Segurança

1. Gerenciamento de Chaves

- **Armazenar com Segurança**: Guarde a chave privada em local seguro, nunca no repositório Git
- **Permissões Restritas**: Use `chmod 400` para restringir acesso à chave
- **Backup**: Faça backup da chave em local seguro
- **Rotação**: Considere criar novas chaves periodicamente

2. Grupos de Segurança

- **Princípio do Menor Privilégio**: Abra apenas as portas necessárias
- **Restrição de IP**: Quando possível, restrinja SSH a IPs específicos em vez de 0.0.0.0/0
- **Segregação**: Use grupos de segurança diferentes para diferentes tipos de instâncias

3. Atualizações e Patches

- **Atualizações Regulares**: Execute `sudo apt update && sudo apt upgrade` regularmente
- **Monitoramento de Segurança**: Habilite AWS Systems Manager Patch Manager
- **Logs**: Monitore logs de sistema para atividades suspeitas

4. Backups

- **Snapshots EBS**: Crie snapshots regulares do volume EBS
- **AMI**: Crie imagens personalizadas (AMI) da instância configurada
- **Dados Críticos**: Use S3 para backup de dados críticos

5. Monitoramento

- **CloudWatch Alarms**: Configure alertas para CPU, memória e disco

- **VPC Flow Logs**: Monitore tráfego de rede
- **CloudTrail**: Rastreie ações de API na AWS

Custo e Otimização

Modelo de Preço do EC2

O EC2 oferece vários modelos de preço:

Modelo	Descrição	Economia
On-Demand	Pague por hora de uso, sem compromisso	Padrão
Reserved Instances	Compromisso de 1 ou 3 anos	Até 72% de desconto
Spot Instances	Capacidade não utilizada com desconto	Até 90% de desconto
Savings Plans	Compromisso de gasto em computação	Até 72% de desconto

Estimativa de Custo para t2.large

A instância **t2.large** em **us-east-1** custa aproximadamente:

- **On-Demand**: ~$0.0928 por hora$ (67/mês)
- **Reserved (1 ano)**: ~$0.0262 por hora$ (19/mês)
- **Spot**: ~$0.0278 por hora$ (20/mês)

Nota: Preços variam por região e podem mudar. Consulte AWS Pricing para valores atualizados.

Dicas de Otimização

1. **Parar Instâncias Não Utilizadas**: Parar (não deletar) economiza custos
2. **Usar Instâncias Apropriadas**: t2.large é suficiente para a maioria das aplicações web
3. **Monitorar Uso**: Use CloudWatch para identificar instâncias subutilizadas

4. **Considerar Reserved Instances**: Se a instância será usada continuamente, considere RI

Conclusão

Amazon EC2 fornece uma plataforma flexível e escalável para hospedar a aplicação Apex Motors. A configuração com instância t2.large em Ubuntu oferece excelente equilíbrio entre desempenho e custo, com capacidade de escalar conforme a demanda cresce.

A segurança é fundamental em qualquer implantação na nuvem. Seguindo as boas práticas de gerenciamento de chaves, configuração de grupos de segurança, atualizações regulares e monitoramento contínuo, sua instância EC2 permanecerá segura e confiável.

O EC2, combinado com o banco de dados PostgreSQL no Render, fornece uma arquitetura robusta e escalável para o projeto Apex Motors, permitindo crescimento futuro sem necessidade de redesenho significativo da infraestrutura.

Docker - Plataforma de Containerização

Introdução ao Docker

Docker Logo

Docker é uma plataforma de código aberto que automatiza o desenvolvimento, envio e execução de aplicações usando contêineres. Ele empacota o código com todas as bibliotecas e dependências necessárias, garantindo que o software funcione de forma consistente e rápida em qualquer ambiente, seja no laptop local, servidores ou nuvem.

Docker revolucionou a forma como desenvolvedores e operações trabalham, eliminando o problema clássico: "funciona na minha máquina, mas não funciona em produção". Com Docker, você cria uma vez e executa em qualquer lugar.

Por que Docker é Importante

- **Consistência**: Mesma aplicação funciona identicamente em desenvolvimento, teste e produção
- **Eficiência**: Contêineres são muito mais leves que máquinas virtuais
- **Velocidade**: Deploy rápido e escalabilidade instantânea
- **Isolamento**: Cada contêiner é independente, evitando conflitos de dependências
- **Comunidade**: Milhões de imagens prontas disponíveis no Docker Hub

---

Conceitos Fundamentais

Contêineres vs. Máquinas Virtuais

A principal diferença entre Docker e máquinas virtuais é a arquitetura:

Aspecto	Máquina Virtual	Contêiner Docker
Sistema Operacional	SO completo para cada aplicação	Compartilha kernel do SO hospedeiro
Tamanho	Gigabytes (GB)	Megabytes (MB)
Tempo de Inicialização	Minutos	Segundos
Isolamento	Completo	Processo isolado
Overhead	Alto	Baixo
Densidade	Poucas VMs por servidor	Centenas de contêineres

Exemplo Prático: Uma máquina virtual Ubuntu pode ocupar 2-5 GB, enquanto um contêiner Docker com Ubuntu ocupa apenas 70-100 MB.

Componentes Principais do Docker

1. Docker Engine

O **Docker Engine** é o motor principal instalado no servidor para criar e gerenciar os contêineres. Ele consiste em:

- **Docker Daemon**: Serviço que roda em background gerenciando contêineres
- **Docker CLI**: Interface de linha de comando para interagir com o daemon
- **REST API**: Interface programática para controlar Docker

2. Imagens Docker

Uma **imagem Docker** é um template (molde) que contém:

- Sistema operacional base (ex: Ubuntu, Alpine)
- Dependências e bibliotecas
- Código da aplicação
- Configurações de ambiente
- Instruções de inicialização

Imagens são **imutáveis** e servem como base para criar contêineres.

3. Contêineres

Um **contêiner** é uma instância em execução de uma imagem Docker. Ele é:

- **Isolado**: Tem seu próprio sistema de arquivos, rede e processos
- **Efêmero**: Pode ser criado e destruído rapidamente
- **Reproduzível**: Sempre executa da mesma forma
- **Portável**: Funciona em qualquer lugar que Docker esteja instalado

4. Dockerfile

Um **Dockerfile** é um arquivo de texto com instruções para criar uma imagem Docker. Exemplo:

```
FROM ubuntu:22.04

RUN apt-get update && apt-get install -y python3 python3-pip

WORKDIR /app

COPY . /app

RUN pip install -r requirements.txt

CMD ["python3", "app.py"]
```

5. Docker Hub

Docker Hub é um repositório central onde desenvolvedores podem:

- Encontrar imagens prontas de aplicações populares
- Compartilhar suas próprias imagens
- Colaborar em projetos open-source
- Armazenar imagens privadas

Exemplos de imagens populares:

- `ubuntu`: Sistema operacional Ubuntu

- `python:3.11`: Python com versão específica
- `postgres:15`: Banco de dados PostgreSQL
- `nginx`: Servidor web Nginx
- `node:18`: Node.js com npm

6. Docker Compose

Docker Compose é uma ferramenta para definir e executar múltiplos contêineres. Usa um arquivo `docker-compose.yaml` para configurar:

- Múltiplos serviços (aplicação, banco de dados, cache)
- Volumes (armazenamento persistente)
- Redes (comunicação entre contêineres)
- Variáveis de ambiente
- Limites de recursos

Instalação do Docker

Pré-requisitos

- Sistema operacional: Linux (Ubuntu, CentOS), Windows, macOS
- Acesso sudo ou privilégios administrativos
- Conexão com internet

Passo a Passo de Instalação no Ubuntu

Passo 1: Atualizar a Lista de Pacotes

```
sudo apt-get update
```

Este comando atualiza o índice de pacotes disponíveis nos repositórios configurados.

Passo 2: Instalar Pacotes Necessários

```
sudo apt-get install ca-certificates curl gnupg lsb-release
```

Estes pacotes são necessários para adicionar um novo repositório via HTTPS:

- `ca-certificates`: Certificados de autoridade de certificação
- `curl`: Ferramenta para fazer requisições HTTP
- `gnupg`: Ferramenta de criptografia GPG
- `lsb-release`: Informações sobre a versão do Ubuntu

Passo 3: Criar Diretório para Chaves

```
sudo mkdir -m 0755 -p /etc/apt/keyrings
```

Este diretório armazenará as chaves GPG dos repositórios.

Passo 4: Adicionar Chave GPG do Docker

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor
-o /usr/share/keyrings/docker-archive-keyring.gpg
```

Este comando baixa a chave GPG do Docker e a armazena localmente para verificar a autenticidade dos pacotes.

Passo 5: Adicionar Repositório do Docker

```
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/usr/share/keyrings/docker-archive-keyring.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo
tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Este comando adiciona o repositório oficial do Docker às fontes de pacotes do APT.

Passo 6: Atualizar Lista de Pacotes Novamente

```
sudo apt-get update
```

Atualiza a lista de pacotes para incluir os do repositório Docker recém-adicionado.

Passo 7: Instalar Docker e Componentes

```
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Instala:

- `docker-ce`: Docker Community Edition (engine principal)
- `docker-ce-cli`: Interface de linha de comando
- `containerd.io`: Runtime de contêineres
- `docker-buildx-plugin`: Plugin para builds avançados
- `docker-compose-plugin`: Plugin Docker Compose

Passo 8: Verificar Instalação

```
docker --version
docker compose version
```

Configuração do Airflow com Docker Compose

Apache Airflow é uma plataforma de orquestração de workflows que permite definir, agendar e monitorar pipelines de dados complexos. Vamos configurá-lo usando Docker Compose.

Passo 1: Baixar docker-compose.yaml do Airflow

```
curl -LfO 'https://airflow.apache.org/docs/apache-airflow/stable/docker-compose.yaml'
```

Este comando baixa o arquivo `docker-compose.yaml` oficial do Airflow, que define todos os serviços necessários:

- **airflow-webserver**: Interface web do Airflow
- **airflow-scheduler**: Agendador de workflows
- **airflow-worker**: Executor de tarefas
- **postgres**: Banco de dados para metadados
- **redis**: Cache para fila de tarefas

Passo 2: Criar Diretórios para DAGs, Logs e Plugins

```
mkdir -p ./dags ./logs ./plugins
```

Cria a estrutura de diretórios:

- `dags/` : Diretório para armazenar seus workflows (DAGs - Directed Acyclic Graphs)
- `logs/` : Diretório para logs de execução
- `plugins/` : Diretório para plugins customizados do Airflow

Passo 3: Criar Arquivo .env com UID do Usuário

```
echo -e "AIRFLOW_UID=$(id -u)" > .env
```

Este arquivo define a variável `AIRFLOW_UID` com o ID do usuário atual. Docker usa isso para definir permissões corretas nos volumes compartilhados, evitando problemas de acesso.

Passo 4: Inicializar o Airflow

```
sudo docker compose up airflow-init
```

Este comando:

- Cria o banco de dados PostgreSQL
- Inicializa as tabelas necessárias
- Cria o usuário administrativo padrão (admin/airflow)
- Prepara o ambiente

Aguarde até que o comando termine com sucesso.

Passo 5: Subir o Airflow em Modo Desacoplado

```
sudo docker compose up -d
```

O flag `-d` significa “detached mode”, ou seja, os contêineres rodam em background.

Passo 6: Aguardar até Tudo Estar Healthy

```
sudo docker ps
```

Verifique o status dos contêineres. A coluna `STATUS` deve mostrar `healthy` para todos os serviços. Pode levar alguns minutos para todos ficarem prontos.

Exemplo de saída esperada:

```
CONTAINER ID   IMAGE                   STATUS
abc123def456   apache/airflow:2.5.0    Up 2 minutes (healthy)
def456ghi789   postgres:13             Up 2 minutes (healthy)
ghi789jkl012   redis:7                 Up 2 minutes (healthy)
```

Acessando a Interface Web do Airflow

Obter o Endereço IP Público

Se estiver usando EC2 na AWS, copie o **Public IPv4 DNS** da instância. Exemplo:

```
ec2-34-224-56-28.compute-1.amazonaws.com
```

Acessar a Interface Web

Abra seu navegador e acesse:

```
http://ec2-34-224-56-28.compute-1.amazonaws.com:8080/
```

Credenciais padrão:

- **Usuário**: admin
- **Senha**: airflow

Configuração do Airflow

Desabilitar Exemplos (Opcional)

O Airflow vem com DAGs de exemplo que podem poluir a interface. Para desabilitá-los:

Passo 1: Editar docker-compose.yaml

```
nano /home/ubuntu/docker-compose.yaml
```

Passo 2: Localizar e Modificar a Variável

Procure pela linha:

```
AIRFLOW__CORE__LOAD_EXAMPLES: 'true'
```

Mude para:

```
AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
```

Passo 3: Salvar e Sair

- Pressione `Ctrl + O` para salvar
- Pressione `Enter` para confirmar
- Pressione `Ctrl + X` para sair

Passo 4: Reiniciar os Contêineres

```
sudo docker compose stop
sudo docker compose up -d
```

Passo 5: Verificar Status

```
sudo docker ps
```

Aguarde até que todos os contêineres estejam com status `healthy`.

Comandos Docker Úteis

Gerenciamento de Contêineres

```
# Listar contêineres em execução
docker ps

# Listar todos os contêineres (incluindo parados)
docker ps -a

# Parar um contêiner
docker stop <container_id>

# Iniciar um contêiner parado
docker start <container_id>

# Remover um contêiner
docker rm <container_id>

# Ver logs de um contêiner
docker logs <container_id>

# Executar comando dentro de um contêiner
docker exec -it <container_id> bash
```

Gerenciamento de Imagens

```
# Listar imagens locais
docker images

# Baixar uma imagem do Docker Hub
docker pull ubuntu:22.04

# Remover uma imagem
docker rmi <image_id>

# Construir uma imagem a partir de Dockerfile
docker build -t minha-app:1.0 .
```

Docker Compose

```
# Subir serviços em background
docker compose up -d

# Parar serviços
docker compose stop

# Remover serviços e volumes
docker compose down

# Ver logs de um serviço
docker compose logs -f <service_name>

# Executar comando em um serviço
docker compose exec <service_name> bash
```

Boas Práticas com Docker

1. Usar Imagens Oficiais

Sempre prefira imagens oficiais do Docker Hub:

```
# ✅ Bom
FROM python:3.11-slim

# ❌ Evitar
FROM minha-imagem-customizada:latest
```

2. Minimizar Tamanho de Imagens

Use imagens base menores (Alpine, Slim):

```
# ✅ Menor (50 MB)
FROM python:3.11-alpine

# ❌ Maior (900 MB)
FROM python:3.11
```

3. Usar .dockerignore

Crie um arquivo `.dockerignore` para excluir arquivos desnecessários:

```
.git
.gitignore
node_modules
*.log
.env
```

4. Não Rodar como Root

Configure um usuário não-root no Dockerfile:

```
RUN useradd -m appuser
USER appuser
```

5. Usar Variáveis de Ambiente

Não hardcode configurações:

```
ENV DATABASE_URL=postgresql://localhost/mydb
ENV DEBUG=false
```

6. Implementar Health Checks

Defina verificações de saúde:

```
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:8000/health || exit 1
```

Solução de Problemas

Contêiner não inicia

```
# Ver logs detalhados
docker logs <container_id>

# Verificar recursos disponíveis
docker stats
```

Permissão negada ao executar docker

```
# Adicionar usuário ao grupo docker
sudo usermod -aG docker $USER

# Aplicar mudanças de grupo
newgrp docker
```

Porta já em uso

```
# Encontrar processo usando a porta
sudo lsof -i :8080

# Matar o processo
sudo kill -9 <PID>
```

Espaço em disco insuficiente

```
# Limpar imagens não utilizadas
docker image prune -a

# Limpar contêineres parados
docker container prune

# Limpar volumes não utilizados
docker volume prune
```

Conclusão

Docker é uma tecnologia fundamental para desenvolvimento e deploy moderno de aplicações. Ao containerizar a aplicação Apex Motors e o Airflow, você garante:

- **Consistência**: Mesma configuração em desenvolvimento e produção
- **Escalabilidade**: Fácil adicionar mais instâncias conforme necessário
- **Manutenibilidade**: Atualizações e patches simplificados
- **Isolamento**: Cada serviço roda independentemente

A combinação de Docker, EC2 e Render fornece uma arquitetura robusta e escalável para o projeto Apex Motors, permitindo orquestração eficiente de workflows de dados e deploy confiável da aplicação.

Apache Airflow - Plataforma de Orquestração de Workflows

Introdução ao Apache Airflow

Apache Airflow Logo

Apache Airflow é uma plataforma open-source de orquestração de fluxos de trabalho (workflows) desenvolvida originalmente pelo Airbnb e hoje parte da Apache Software Foundation. Ele permite agendar, monitorar e gerenciar pipelines de dados complexos (ETL/ELT) utilizando código Python, organizando tarefas em grafos acíclicos direcionados (DAGs).

Airflow revolucionou a forma como engenheiros de dados automatizam processos, substituindo soluções tradicionais como cron jobs por uma plataforma robusta, escalável e com interface visual intuitiva.

Por que Usar Airflow

- **Código como Configuração**: Workflows definidos em Python, versionáveis e testáveis
- **Visualização Completa**: Interface gráfica mostrando status, dependências e logs
- **Escalabilidade**: Suporta execução distribuída de tarefas
- **Flexibilidade**: Integração nativa com centenas de tecnologias
- **Recuperação de Falhas**: Reexecução automática e tratamento de erros
- **Comunidade Ativa**: Mantido por comunidade global com atualizações regulares

---

Principais Características e Benefícios

Característica	Descrição
Definição em Python	Os fluxos de trabalho são definidos como código, tornando-os dinâmicos e flexíveis
Orquestração de DAGs	Utiliza Grafos Acíclicos Direcionados (DAGs) para gerenciar dependências de tarefas
Interface Gráfica (UI)	Oferece visualização completa para monitorar o status das tarefas, agendamentos e logs em tempo real
Escalabilidade e Flexibilidade	Suporta integrações nativas com diversas tecnologias, incluindo nuvens como AWS, GCP e Azure
Recuperação de Falhas	Permite reexecutar tarefas automaticamente a partir do ponto de falha
Monitoramento Avançado	Alertas, notificações e histórico completo de execuções
Controle de Acesso	Gerenciamento de usuários, roles e permissões
Backfill de Dados	Execução retroativa de workflows para períodos passados

---

Conceitos Fundamentais

DAG (Directed Acyclic Graph)

Um **DAG** é a estrutura fundamental do Airflow. Representa um workflow como um grafo onde:

- **Nós** representam tarefas (tasks)
- **Arestas** representam dependências entre tarefas
- **Acíclico** significa que não há loops (uma tarefa não pode depender de si mesma)
- **Direcionado** significa que há uma direção clara do fluxo

Exemplo Visual:

```
    [Tarefa A]
       |
       v
    [Tarefa B]
     /       \
    v         v
[Tarefa C] [Tarefa D]
    \         /
     v       v
    [Tarefa E]
```

Neste exemplo, a Tarefa A deve ser executada primeiro, depois B, depois C e D em paralelo, e finalmente E.

Tasks (Tarefas)

Uma **task** é uma unidade de trabalho que será executada. Tipos comuns:

- **PythonOperator**: Executa função Python
- **BashOperator**: Executa comando bash
- **PostgresOperator**: Executa query SQL no PostgreSQL
- **SnowflakeOperator**: Executa query no Snowflake
- **S3Operator**: Interage com buckets S3
- **EmailOperator**: Envia emails

Operators

Um **operator** define o tipo de trabalho que uma task realiza. Exemplos:

```python
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator
from airflow.providers.postgres.operators.postgres import PostgresOperator

# Operator Python
task1 = PythonOperator(
    task_id='minha_tarefa_python',
    python_callable=minha_funcao
)

# Operator Bash
task2 = BashOperator(
    task_id='minha_tarefa_bash',
    bash_command='echo "Olá Mundo"'
)

# Operator PostgreSQL
task3 = PostgresOperator(
    task_id='minha_query_postgres',
    sql='SELECT * FROM tabela',
    postgres_conn_id='postgres'
)
```

Hooks

Hooks são interfaces para conectar a sistemas externos. Exemplos:

- **PostgresHook**: Conexão com PostgreSQL
- **SnowflakeHook**: Conexão com Snowflake
- **S3Hook**: Acesso a buckets S3
- **MySqlHook**: Conexão com MySQL
- **HttpHook**: Requisições HTTP

```python
from airflow.providers.postgres.hooks.postgres import PostgresHook

hook = PostgresHook(postgres_conn_id='postgres')
conn = hook.get_conn()
cursor = conn.cursor()
cursor.execute('SELECT * FROM veiculos')
```

Componentes Principais do Airflow

1. Scheduler (Agendador)

O **Scheduler** é responsável por:

- Monitorar DAGs e determinar quando executar
- Criar instâncias de tarefas (task instances)
- Gerenciar dependências
- Agendar execuções conforme o intervalo definido

```bash
# Iniciar scheduler
airflow scheduler
```

2. Webserver (Servidor Web)

O **Webserver** fornece a interface gráfica onde você pode:

- Visualizar DAGs e suas dependências
- Monitorar execuções em tempo real
- Ver logs de tarefas
- Gerenciar conexões e variáveis
- Controlar usuários e permissões

```
# Iniciar webserver
airflow webserver --port 8080
```

3. Worker (Executor)

O **Worker** executa as tarefas. Existem diferentes tipos de executores:

- **SequentialExecutor**: Executa tarefas uma por uma (desenvolvimento)
- **LocalExecutor**: Executa tarefas em paralelo localmente
- **CeleryExecutor**: Executa tarefas distribuídas em múltiplas máquinas
- **KubernetesExecutor**: Executa tarefas em pods Kubernetes

4. Metadata Database

O banco de dados armazena:

- Definições de DAGs
- Status de execuções
- Logs
- Conexões e variáveis
- Histórico de tarefas

Geralmente usa PostgreSQL em produção.

5. Message Broker (Opcional)

Necessário para executores distribuídos como CeleryExecutor. Exemplos:

- **RabbitMQ**: Message broker robusto
- **Redis**: Cache e message broker

Criação de uma DAG - Passo a Passo

Passo 1: Conectar à Instância EC2

```
# Terminal local
cd Desktop\apex_motors

# Conectar via SSH
ssh -i "key_pair_apex_motors.pem" ubuntu@ec2-100-26-23-29.compute-1.amazonaws.com
```

Passo 2: Navegar até o Diretório de DAGs

```
# Na instância EC2
ls
cd dags/
```

Passo 3: Criar Arquivo da DAG

```
touch apexmotors.py
ls
```

Passo 4: Editar o Arquivo

```
nano apexmotors.py
```

Passo 5: Colar o Código da DAG

Copie e cole o código abaixo no editor nano.

Passo 6: Salvar e Sair

- Pressione `Ctrl + X`
- Pressione `Y` para confirmar
- Pressione `Enter` para salvar

Passo 7: Verificar o Arquivo

```
cat apexmotors.py
```

Exemplo Prático: ETL PostgreSQL para Snowflake

Descrição da DAG

Esta DAG realiza um pipeline ETL que:

1. Extrai dados incrementalmente do PostgreSQL
2. Carrega os dados no Snowflake
3. Executa diariamente

As tabelas processadas são:

- veiculos
- estados
- cidades
- concessionarias
- vendedores
- clientes
- vendas

Código da DAG

```
from datetime import datetime, timedelta
from airflow.decorators import dag, task
from airflow.providers.postgres.hooks.postgres import PostgresHook
from airflow.providers.snowflake.hooks.snowflake import SnowflakeHook

# Argumentos padrão para todas as tarefas
default_args = {
    'owner': 'airflow',
    'depends_on_past': False,
    'start_date': datetime(2024, 1, 1),
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 0,
    'retry_delay': timedelta(minutes=1),
}

# Decorador DAG
@dag(
    dag_id='postgres_to_snowflake',
    default_args=default_args,
    description='Load data incrementally from Postgres to Snowflake',
    schedule_interval=timedelta(days=1),
    catchup=False
)
def postgres_to_snowflake_etl():
    # Lista de tabelas a processar
    table_names = ['veiculos', 'estados', 'cidades', 'concessionarias',
                   'vendedores', 'clientes', 'vendas']

    for table_name in table_names:
        # Tarefa 1: Obter ID máximo do Snowflake
        @task(task_id=f'get_max_id_{table_name}')
        def get_max_primary_key(table_name: str):
            """
            Conecta ao Snowflake e obtém o ID máximo da tabela.
            Isso permite carregar apenas registros novos (incrementalmente).
            """
            with SnowflakeHook(snowflake_conn_id='snowflake').get_conn() as
conn:
                with conn.cursor() as cursor:
                    cursor.execute(f"SELECT MAX(ID_{table_name}) FROM
{table_name}")
                    max_id = cursor.fetchone()[0]
```

```
                    return max_id if max_id is not None else 0

        # Tarefa 2: Carregar dados incrementalmente
        @task(task_id=f'load_data_{table_name}')
        def load_incremental_data(table_name: str, max_id: int):
            """
            Conecta ao PostgreSQL, busca registros com ID > max_id,
            e insere no Snowflake.
            """
            # Conexão com PostgreSQL
            with PostgresHook(postgres_conn_id='postgres').get_conn() as
pg_conn:
                with pg_conn.cursor() as pg_cursor:
                    primary_key = f'ID_{table_name}'

                    # Obter nomes de colunas dinamicamente
                    pg_cursor.execute(
                        f"SELECT column_name FROM information_schema.columns
"
                        f"WHERE table_name = '{table_name}'"
                    )
                    columns = [row[0] for row in pg_cursor.fetchall()]
                    columns_list_str = ', '.join(columns)
                    placeholders = ', '.join(['%s'] * len(columns))

                    # Buscar dados novos do PostgreSQL
                    pg_cursor.execute(
                        f"SELECT {columns_list_str} FROM {table_name} "
                        f"WHERE {primary_key} > {max_id}"
                    )
                    rows = pg_cursor.fetchall()

                    # Inserir dados no Snowflake
                    with
SnowflakeHook(snowflake_conn_id='snowflake').get_conn() as sf_conn:
                        with sf_conn.cursor() as sf_cursor:
                            insert_query = (
                                f"INSERT INTO {table_name}
({columns_list_str}) "
                                f"VALUES ({placeholders})"
                            )
                            for row in rows:
                                sf_cursor.execute(insert_query, row)

        # Definir dependências: get_max_id deve executar antes de load_data
        max_id = get_max_primary_key(table_name)
```

```
        load_incremental_data(table_name, max_id)

# Instanciar a DAG
postgres_to_snowflake_etl_dag = postgres_to_snowflake_etl()
```

Explicação do Código

Imports

```
from datetime import datetime, timedelta
from airflow.decorators import dag, task
from airflow.providers.postgres.hooks.postgres import PostgresHook
from airflow.providers.snowflake.hooks.snowflake import SnowflakeHook
```

Importa as bibliotecas necessárias para criar DAGs e conectar aos bancos de dados.

Default Args

```
default_args = {
    'owner': 'airflow',                    # Proprietário da DAG
    'depends_on_past': False,              # Não depende de execuções
passadas
    'start_date': datetime(2024, 1, 1),    # Quando começar a agendar
    'email_on_failure': False,             # Não enviar email em falhas
    'email_on_retry': False,               # Não enviar email em
retentativas
    'retries': 0,                          # Não tentar novamente em caso de
falha
    'retry_delay': timedelta(minutes=1),   # Aguardar 1 minuto antes de
retentar
}
```

Decorador DAG

```
@dag(
    dag_id='postgres_to_snowflake',                    # ID único da DAG
    default_args=default_args,                         # Argumentos padrão
    description='Load data incrementally from Postgres to Snowflake',
    schedule_interval=timedelta(days=1),               # Executar
diariamente
    catchup=False                                      # Não executar
períodos passados
)
```

Define a DAG com agendamento diário.

Loop de Tabelas

```
for table_name in table_names:
    @task(task_id=f'get_max_id_{table_name}')
    def get_max_primary_key(table_name: str):
        # ...
```

Para cada tabela, cria tarefas específicas (get_max_id_veiculos, load_data_veiculos, etc).

Definir Dependências

```
max_id = get_max_primary_key(table_name)
load_incremental_data(table_name, max_id)
```

O resultado de `get_max_primary_key` é passado como argumento para `load_incremental_data`, criando uma dependência.

Configurando Conexões no Airflow

Adicionar Conexão PostgreSQL

1. Acesse a interface web do Airflow: `http://seu-ip:8080`
2. Clique em **Admin** → **Connections**
3. Clique em **Create**
4. Preencha:
    - **Conn Id**: postgres
    - **Conn Type**: Postgres
    - **Host**: dpg-d5a8u3je5dus73avg4cd-s (do Render)
    - **Port**: 5432
    - **Database**: apex_db_5nni
    - **Login**: apex_db_5nni_user
    - **Password**: [sua senha]
5. Clique em **Save**

Adicionar Conexão Snowflake

1. Clique em **Admin** → **Connections**
2. Clique em **Create**
3. Preencha:
    - **Conn Id**: snowflake
    - **Conn Type**: Snowflake
    - **Account**: [seu account ID]
    - **Warehouse**: [seu warehouse]
    - **Database**: [seu database]
    - **Schema**: [seu schema]
    - **Login**: [seu usuário]
    - **Password**: [sua senha]

4. Clique em **Save**

Monitoramento e Troubleshooting

Visualizar DAG na Interface Web

1. Acesse `http://seu-ip:8080`
2. Clique em **DAGs**
3. Clique em **postgres_to_snowflake**
4. Veja o gráfico de dependências na aba **Graph**

Ver Logs de Execução

1. Na página da DAG, clique em **Tree** ou **Graph**
2. Clique em uma tarefa
3. Clique em **Logs**
4. Veja os logs detalhados da execução

Reexecutar uma Tarefa

1. Na página da DAG, clique em uma tarefa que falhou
2. Clique em **Clear**
3. A tarefa será reexecutada na próxima execução do scheduler

Problemas Comuns

Conexão Recusada

Problema: “Connection refused” ao conectar ao PostgreSQL

Solução: Verifique se as credenciais estão corretas e se o banco está acessível.

Timeout

Problema: Tarefa excede o tempo limite

Solução: Aumente o `execution_timeout` na tarefa:

```
@task(task_id='minha_tarefa', execution_timeout=timedelta(hours=1))
def minha_funcao():
    pass
```

Memória Insuficiente

Problema: Erro de memória ao processar muitos dados

Solução: Processe dados em lotes:

```
# Em vez de carregar tudo de uma vez
for batch in get_data_in_batches(1000):
    insert_to_snowflake(batch)
```

Boas Práticas com Airflow

1. Idempotência

Tarefas devem produzir o mesmo resultado quando executadas múltiplas vezes:

```python
# ✅ Bom - Idempotente
@task
def load_data(table_name: str, max_id: int):
    # DELETE + INSERT garante idempotência
    cursor.execute(f"DELETE FROM {table_name} WHERE id > {max_id}")
    cursor.execute(f"INSERT INTO {table_name} SELECT * FROM source WHERE id
> {max_id}")

# ❌ Ruim - Não idempotente
@task
def load_data(table_name: str):
    # Pode inserir duplicatas se executado múltiplas vezes
    cursor.execute(f"INSERT INTO {table_name} SELECT * FROM source")
```

2. Logging Adequado

Adicione logs para facilitar troubleshooting:

```python
from airflow.utils.log.logging_mixin import LoggingMixin

@task
def minha_tarefa():
    logger = LoggingMixin().log
    logger.info(f"Iniciando processamento")
    logger.warning(f"Aviso importante")
    logger.error(f"Erro encontrado")
```

3. Tratamento de Erros

Implemente tratamento robusto:

```python
@task(retries=3, retry_delay=timedelta(minutes=5))
def minha_tarefa():
    try:
        # Código principal
        pass
    except Exception as e:
        logger.error(f"Erro: {str(e)}")
        raise
```

4. Evitar Hardcoding

Use variáveis e conexões:

```python
# ✅ Bom
postgres_conn_id = 'postgres'
hook = PostgresHook(postgres_conn_id=postgres_conn_id)

# ❌ Ruim
hook = PostgresHook(postgres_conn_id='postgresql://user:pass@host/db')
```

5. Documentação

Documente suas DAGs:

```
@dag(
    dag_id='minha_dag',
    description='Descrição clara do que a DAG faz',
    doc_md="""
    # Minha DAG

    Esta DAG realiza:
    - Extração de dados do PostgreSQL
    - Transformação dos dados
    - Carregamento no Snowflake

    **Agendamento**: Diariamente às 2:00 AM
    **Proprietário**: Data Engineering Team
    """
)
```

Conclusão

Apache Airflow é a solução ideal para orquestração de pipelines de dados complexos no projeto Apex Motors. Ao combinar Airflow com PostgreSQL e Snowflake, você cria uma arquitetura robusta de ETL que:

- **Automatiza** processos de dados repetitivos
- **Monitora** execuções em tempo real
- **Recupera** de falhas automaticamente
- **Escala** conforme a demanda cresce

A DAG de exemplo demonstra como extrair dados incrementalmente do PostgreSQL e carregá-los no Snowflake, um padrão comum em pipelines de dados modernos.

Com Docker, EC2, Render e Airflow, você possui uma stack completa de engenharia de dados pronta para produção.

Google BigQuery - Data Warehouse na Nuvem

Introdução ao Google BigQuery

Google BigQuery Logo

Google BigQuery é um serviço de data warehouse totalmente gerenciado (NoOps) oferecido pelo Google Cloud Platform (GCP). Ele permite analisar grandes volumes de dados na escala de petabytes quase em tempo real, utilizando SQL padrão, sem necessidade de gerenciar infraestrutura, servidores ou administradores de banco de dados.

BigQuery é a escolha ideal para empresas que precisam de análise de dados em tempo real, inteligência de negócios (BI), machine learning e processamento de grandes volumes de dados sem complexidade operacional.

Por que Usar BigQuery

- **Sem Gerenciamento de Infraestrutura**: Totalmente gerenciado (NoOps) - você se concentra nos dados, não na infraestrutura
- **Velocidade Extrema**: Processa terabytes em segundos e petabytes em minutos
- **Escalabilidade Ilimitada**: Cresce automaticamente conforme suas necessidades
- **SQL Padrão**: Utiliza SQL familiar, facilitando a migração de outros bancos
- **Machine Learning Integrado**: BigQuery ML permite criar modelos sem sair do SQL
- **Custo-Benefício**: Modelo de preço flexível com 1 TB de consultas gratuitas por mês
- **Integração com Google Cloud**: Conecta-se facilmente com Analytics, Data Studio, Looker, etc.

Principais Características e Benefícios

Característica	Descrição
Totalmente Gerenciado (NoOps)	Não é necessário configurar servidores, atualizar software ou gerenciar administradores de banco de dados
Velocidade e Escala	Processa consultas em terabytes de dados em segundos e petabytes em minutos, utilizando o sistema de arquivos distribuído Colossus e o motor Dremel
SQL Padrão	Utiliza linguagem SQL, facilitando a consulta de dados estruturados e semiestruturados
Machine Learning Integrado	BigQuery ML permite criar e executar modelos de aprendizado de máquina diretamente com SQL
Armazenamento Colunar	Armazena dados em colunas para consultas analíticas mais rápidas e eficientes
Custo-Benefício	Modelos de preços flexíveis (sob demanda ou capacidade fixa), incluindo 1 TB de consultas gratuitas por mês
Integração	Conecta-se facilmente a outras ferramentas do Google Cloud, como Google Analytics 4, e suporta consultas federadas
Segurança e Conformidade	Criptografia em repouso e em trânsito, conformidade com HIPAA, SOC 2, ISO 27001
Backup e Recuperação	Retenção automática de snapshots de tabelas por 7 dias

---

Conceitos Fundamentais

Dataset

Um **dataset** é um container que organiza e controla o acesso às tabelas. Funciona como um esquema em bancos de dados tradicionais.

```
Projeto GCP
    └── Dataset (apex_motors)
            ├── Tabela: veiculos
            ├── Tabela: estados
            ├── Tabela: cidades
            ├── Tabela: concessionarias
            ├── Tabela: vendedores
            ├── Tabela: clientes
            └── Tabela: vendas
```

Tabelas

Uma **tabela** armazena dados em linhas e colunas. BigQuery suporta:

- **Tabelas Padrão**: Armazenam dados permanentemente
- **Tabelas Externas**: Referenciam dados em Cloud Storage, Bigtable, etc.
- **Snapshots**: Cópias de tabelas em um ponto no tempo
- **Visualizações (Views)**: Consultas salvas que atuam como tabelas virtuais

Tipos de Dados

BigQuery suporta diversos tipos de dados:

Tipo	Descrição	Exemplo
INTEGER	Números inteiros de 64 bits	12345
FLOAT64	Números de ponto flutuante	123.45
DECIMAL	Números decimais de precisão fixa	DECIMAL(10,2)
STRING	Texto	‘Apex Motors’
BYTES	Dados binários	b’dados’
BOOLEAN	Verdadeiro ou Falso	TRUE, FALSE
DATE	Data (YYYY-MM-DD)	2024-03-01
TIME	Hora (HH:MM:SS)	14:30:00
DATETIME	Data e hora	2024-03-01 14:30:00
TIMESTAMP	Timestamp UTC	2024-03-01T14:30:00Z
ARRAY	Lista de valores	[1, 2, 3]
STRUCT	Estrutura aninhada	STRUCT(nome STRING, idade INT64)
JSON	Dados JSON	{“chave”: “valor”}

Particionamento

Particionamento divide uma tabela em segmentos menores baseado em uma coluna, melhorando performance e reduzindo custos:

```
CREATE TABLE vendas (
    id_vendas INTEGER,
    data_venda TIMESTAMP,
    valor DECIMAL(10,2)
)
PARTITION BY DATE(data_venda);
```

Benefícios:

- Consultas mais rápidas (escaneia apenas partições relevantes)

- Reduz custos (paga apenas pelos dados consultados)
- Melhor gerenciamento de dados

Arquitetura do BigQuery

Motor de Processamento: Dremel

O **Dremel** é o motor de processamento distribuído do BigQuery que:

- Distribui consultas entre milhares de máquinas
- Processa dados em paralelo
- Retorna resultados em segundos

Sistema de Arquivos: Colossus

O **Colossus** é o sistema de armazenamento distribuído que:

- Armazena dados de forma redundante
- Garante alta disponibilidade
- Escala automaticamente

Armazenamento Colunar

BigQuery armazena dados em **colunas** (não em linhas), o que:

- Melhora compressão de dados
- Acelera consultas analíticas
- Reduz custos (paga apenas pelas colunas consultadas)

Criando as Tabelas do Projeto Apex Motors

Tabela: Veículos

Armazena informações sobre os veículos disponíveis para venda.

```
CREATE TABLE veiculos (
   id_veiculos INTEGER,
   nome VARCHAR(255) NOT NULL,
   tipo VARCHAR(100) NOT NULL,
   valor DECIMAL(10, 2) NOT NULL,
   data_atualizacao TIMESTAMP_LTZ,
   data_inclusao TIMESTAMP_LTZ
);
```

Descrição das Colunas:

- `id_veiculos` : Identificador único do veículo (chave primária)
- `nome` : Nome do modelo do veículo (ex: “Apex Ion”, “Apex Vortex”)
- `tipo` : Categoria do veículo (ex: “SUV Urbano/Aventura”, “Sedã Executivo”)
- `valor` : Preço do veículo em reais
- `data_atualizacao` : Quando o registro foi atualizado pela última vez
- `data_inclusao` : Quando o registro foi criado

Tabela: Estados

Armazena informações sobre estados brasileiros.

```
CREATE TABLE estados (
   id_estados INTEGER,
   estado VARCHAR(100) NOT NULL,
   sigla CHAR(2) NOT NULL,
   data_inclusao TIMESTAMP_LTZ,
   data_atualizacao TIMESTAMP_LTZ
);
```

Descrição das Colunas:

- `id_estados` : Identificador único do estado (chave primária)
- `estado` : Nome completo do estado (ex: “São Paulo”, “Rio de Janeiro”)
- `sigla` : Sigla de duas letras (ex: “SP”, “RJ”)
- `data_inclusao` : Quando o registro foi criado
- `data_atualizacao` : Quando o registro foi atualizado pela última vez

Tabela: Cidades

Armazena informações sobre cidades onde há concessionárias.

```
CREATE TABLE cidades (
   id_cidades INTEGER,
   cidade VARCHAR(255) NOT NULL,
   id_estados INTEGER NOT NULL,
   data_inclusao TIMESTAMP_LTZ,
   data_atualizacao TIMESTAMP_LTZ
);
```

Descrição das Colunas:

- `id_cidades` : Identificador único da cidade (chave primária)
- `cidade` : Nome da cidade (ex: “Rio de Janeiro”, “São Paulo”)
- `id_estados` : Referência ao estado (chave estrangeira)
- `data_inclusao` : Quando o registro foi criado
- `data_atualizacao` : Quando o registro foi atualizado pela última vez

Tabela: Concessionárias

Armazena informações sobre as concessionárias Apex Motors.

```sql
CREATE TABLE concessionarias (
   id_concessionarias INTEGER,
   concessionaria VARCHAR(255) NOT NULL,
   id_cidades INTEGER NOT NULL,
   data_inclusao TIMESTAMP_LTZ,
   data_atualizacao TIMESTAMP_LTZ
);
```

Descrição das Colunas:

- `id_concessionarias` : Identificador único da concessionária (chave primária)
- `concessionaria` : Nome da concessionária (ex: “Apex Motors Rio de Janeiro”)
- `id_cidades` : Referência à cidade (chave estrangeira)
- `data_inclusao` : Quando o registro foi criado
- `data_atualizacao` : Quando o registro foi atualizado pela última vez

Tabela: Vendedores

Armazena informações sobre os vendedores das concessionárias.

```sql
CREATE TABLE vendedores (
   id_vendedores INTEGER,
   nome VARCHAR(255) NOT NULL,
   id_concessionarias INTEGER NOT NULL,
   data_inclusao TIMESTAMP_LTZ,
   data_atualizacao TIMESTAMP_LTZ
);
```

Descrição das Colunas:

- `id_vendedores` : Identificador único do vendedor (chave primária)
- `nome` : Nome completo do vendedor
- `id_concessionarias` : Referência à concessionária (chave estrangeira)
- `data_inclusao` : Quando o registro foi criado
- `data_atualizacao` : Quando o registro foi atualizado pela última vez

Tabela: Clientes

Armazena informações sobre os clientes que compraram veículos.

```
CREATE TABLE clientes (
   id_clientes INTEGER,
   cliente VARCHAR(255) NOT NULL,
   endereco TEXT NOT NULL,
   id_concessionarias INTEGER NOT NULL,
   data_inclusao TIMESTAMP_LTZ,
   data_atualizacao TIMESTAMP_LTZ
);
```

Descrição das Colunas:

- `id_clientes`: Identificador único do cliente (chave primária)
- `cliente`: Nome completo do cliente
- `endereco`: Endereço completo do cliente
- `id_concessionarias`: Referência à concessionária (chave estrangeira)
- `data_inclusao`: Quando o registro foi criado
- `data_atualizacao`: Quando o registro foi atualizado pela última vez

Tabela: Vendas

Armazena informações sobre todas as transações de venda.

```
CREATE TABLE vendas (
   id_vendas INTEGER,
   id_veiculos INTEGER NOT NULL,
   id_concessionarias INTEGER NOT NULL,
   id_vendedores INTEGER NOT NULL,
   id_clientes INTEGER NOT NULL,
   valor_pago DECIMAL(10, 2) NOT NULL,
   data_venda TIMESTAMP_LTZ,
   data_inclusao TIMESTAMP_LTZ,
   data_atualizacao TIMESTAMP_LTZ
);
```

Descrição das Colunas:

- `id_vendas` : Identificador único da venda (chave primária)
- `id_veiculos` : Referência ao veículo vendido (chave estrangeira)
- `id_concessionarias` : Referência à concessionária (chave estrangeira)
- `id_vendedores` : Referência ao vendedor (chave estrangeira)
- `id_clientes` : Referência ao cliente (chave estrangeira)
- `valor_pago` : Valor total pago pela venda
- `data_venda` : Data e hora da venda
- `data_inclusao` : Quando o registro foi criado
- `data_atualizacao` : Quando o registro foi atualizado pela última vez

Executando Consultas no BigQuery

Consulta Básica

```sql
-- Selecionar todos os veículos
SELECT
    id_veiculos,
    nome,
    tipo,
    valor
FROM veiculos
LIMIT 10;
```

Consulta com Agregação

```sql
-- Contar quantas vendas foram feitas por concessionária
SELECT
    c.concessionaria,
    COUNT(*) AS total_vendas,
    SUM(v.valor_pago) AS receita_total
FROM vendas v
JOIN concessionarias c ON v.id_concessionarias = c.id_concessionarias
GROUP BY c.concessionaria
ORDER BY receita_total DESC;
```

Consulta com Múltiplos JOINs

```sql
-- Listar todas as vendas com detalhes completos
SELECT
    v.id_vendas,
    cl.cliente,
    ve.nome AS veiculo,
    vd.nome AS vendedor,
    c.concessionaria,
    v.valor_pago,
    v.data_venda
FROM vendas v
JOIN clientes cl ON v.id_clientes = cl.id_clientes
JOIN veiculos ve ON v.id_veiculos = ve.id_veiculos
JOIN vendedores vd ON v.id_vendedores = vd.id_vendedores
JOIN concessionarias c ON v.id_concessionarias = c.id_concessionarias
ORDER BY v.data_venda DESC
LIMIT 100;
```

Consulta com Subconsulta

```sql
-- Encontrar clientes que compraram acima da média
SELECT
    cl.cliente,
    COUNT(*) AS total_compras,
    SUM(v.valor_pago) AS gasto_total
FROM vendas v
JOIN clientes cl ON v.id_clientes = cl.id_clientes
GROUP BY cl.cliente
HAVING SUM(v.valor_pago) > (
    SELECT AVG(valor_pago) FROM vendas
)
ORDER BY gasto_total DESC;
```

Consulta com Window Functions

```sql
-- Ranking de vendedores por receita gerada
SELECT
    vd.nome,
    c.concessionaria,
    SUM(v.valor_pago) AS receita,
    ROW_NUMBER() OVER (PARTITION BY c.concessionaria ORDER BY
SUM(v.valor_pago) DESC) AS ranking
FROM vendas v
JOIN vendedores vd ON v.id_vendedores = vd.id_vendedores
JOIN concessionarias c ON v.id_concessionarias = c.id_concessionarias
GROUP BY vd.nome, c.concessionaria
ORDER BY c.concessionaria, ranking;
```

Integrando BigQuery com Airflow

Instalação do Provedor

```bash
# Na instância EC2
pip install apache-airflow-providers-google
```

Exemplo de DAG com BigQuery

```python
from datetime import datetime, timedelta
from airflow.decorators import dag, task
from airflow.providers.google.cloud.operators.bigquery import (
    BigQueryCreateEmptyTableOperator,
    BigQueryInsertJobOperator
)
from airflow.providers.postgres.hooks.postgres import PostgresHook
from google.cloud import bigquery

default_args = {
    'owner': 'airflow',
    'start_date': datetime(2024, 1, 1),
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

@dag(
    dag_id='postgres_to_bigquery',
    default_args=default_args,
    schedule_interval=timedelta(days=1),
    catchup=False
)
def postgres_to_bigquery_etl():

    @task
    def extract_from_postgres(table_name: str):
        """Extrai dados do PostgreSQL"""
        hook = PostgresHook(postgres_conn_id='postgres')
        conn = hook.get_conn()
        cursor = conn.cursor()

        query = f"SELECT * FROM {table_name}"
        cursor.execute(query)

        columns = [desc[0] for desc in cursor.description]
        rows = cursor.fetchall()

        cursor.close()
        conn.close()

        return {
            'columns': columns,
            'rows': rows
```

```
        }

    @task
    def load_to_bigquery(table_name: str, data: dict):
        """Carrega dados no BigQuery"""
        client = bigquery.Client(project='seu-projeto-gcp')
        dataset_id = 'apex_motors'
        table_id = f'{dataset_id}.{table_name}'

        rows_to_insert = []
        for row in data['rows']:
            row_dict = dict(zip(data['columns'], row))
            rows_to_insert.append(row_dict)

        errors = client.insert_rows_json(table_id, rows_to_insert)

        if errors:
            raise Exception(f"Erros ao inserir dados: {errors}")

        return f"Carregados {len(rows_to_insert)} registros em {table_id}"

    # Processar cada tabela
    tables = ['veiculos', 'estados', 'cidades', 'concessionarias',
              'vendedores', 'clientes', 'vendas']

    for table in tables:
        data = extract_from_postgres(table)
        load_to_bigquery(table, data)

postgres_to_bigquery_etl_dag = postgres_to_bigquery_etl()
```

Boas Práticas com BigQuery

1. Usar Particionamento

Sempre particione tabelas grandes por data ou outra coluna relevante:

```sql
CREATE TABLE vendas (
    id_vendas INTEGER,
    data_venda TIMESTAMP,
    valor DECIMAL(10,2)
)
PARTITION BY DATE(data_venda)
CLUSTER BY id_concessionarias;
```

2. Usar Clustering

Agrupe dados relacionados para melhorar performance:

```sql
CREATE TABLE vendas (
    id_vendas INTEGER,
    id_concessionarias INTEGER,
    id_vendedores INTEGER,
    valor DECIMAL(10,2)
)
CLUSTER BY id_concessionarias, id_vendedores;
```

3. Selecionar Apenas Colunas Necessárias

BigQuery cobra por dados lidos, então selecione apenas o necessário:

```sql
-- ✅ Bom - Seleciona apenas colunas necessárias
SELECT id_vendas, valor_pago FROM vendas;

-- ❌ Ruim - Seleciona todas as colunas
SELECT * FROM vendas;
```

4. Usar Visualizações (Views)

Crie visualizações para consultas complexas reutilizáveis:

```sql
CREATE VIEW vendas_por_concessionaria AS
SELECT
    c.concessionaria,
    COUNT(*) AS total_vendas,
    SUM(v.valor_pago) AS receita
FROM vendas v
JOIN concessionarias c ON v.id_concessionarias = c.id_concessionarias
GROUP BY c.concessionaria;
```

5. Usar Materialized Views

Para consultas frequentes, use visualizações materializadas (cache):

```sql
CREATE MATERIALIZED VIEW vendas_resumo AS
SELECT
    DATE(data_venda) AS data,
    id_concessionarias,
    COUNT(*) AS total_vendas,
    SUM(valor_pago) AS receita
FROM vendas
GROUP BY data, id_concessionarias;
```

6. Monitorar Custos

Use o console do GCP para monitorar:

- Dados lidos por consulta
- Slots utilizados
- Custos estimados

7. Usar Slots para Previsibilidade

Para workloads previsíveis, compre slots (capacidade fixa):

```
Sob Demanda: $6.25 por TB consultado
Slots: $0.04 por slot por hora (100 slots = 1 TB/min)
```

Conectando BigQuery com Ferramentas BI

Google Data Studio

1. Acesse datastudio.google.com
2. Crie um novo relatório
3. Conecte ao BigQuery como fonte de dados
4. Crie visualizações interativas

Looker

1. Configure Looker para conectar ao BigQuery
2. Crie explores e dashboards
3. Compartilhe insights com a organização

Tableau

1. Baixe e instale Tableau
2. Conecte ao BigQuery via Google Cloud
3. Crie visualizações e dashboards

Segurança e Conformidade

Controle de Acesso

BigQuery integra-se com IAM do Google Cloud:

```
Projeto
├── Dataset
│   ├── Tabela
│   └── Visualização
└── Permissões (IAM)
    ├── Editor
    ├── Leitor
    ├── Administrador de Dados
    └── Operador de Trabalho
```

Criptografia

- **Em Repouso**: Criptografia automática com chaves gerenciadas pelo Google
- **Em Trânsito**: Criptografia TLS 1.2+
- **Chaves Gerenciadas pelo Cliente**: Opção disponível para conformidade

Auditoria

BigQuery registra todas as ações em Cloud Audit Logs:

- Quem acessou os dados
- Quando foi acessado
- Que consultas foram executadas
- Quanto foi gasto

Solução de Problemas

Consulta Lenta

Problema: Consulta demora muito tempo

Solução:

- Use `EXPLAIN` para analisar o plano de execução
- Adicione particionamento e clustering

- Reduza o volume de dados consultados
- Use visualizações materializadas

```
EXPLAIN SELECT * FROM vendas WHERE data_venda > '2024-01-01';
```

Limite de Taxa Excedido

Problema: “Quota exceeded” ou “Rate limit exceeded”

Solução:

- Compre slots para capacidade dedicada
- Implemente backoff exponencial em retries
- Distribua consultas ao longo do tempo

Dados Não Aparecem

Problema: Tabela vazia ou dados não aparecem

Solução:

- Verifique permissões IAM
- Confirme que os dados foram carregados
- Verifique se a tabela existe no dataset correto
- Use `SELECT COUNT(*) FROM tabela` para verificar

Conclusão

Google BigQuery é a solução ideal para análise de dados em escala no projeto Apex Motors. Ao combinar BigQuery com PostgreSQL, Snowflake e Airflow, você cria uma arquitetura moderna de data warehouse que:

- **Processa** grandes volumes de dados rapidamente
- **Escala** automaticamente conforme a demanda

- **Integra-se** com ferramentas BI para insights
- **Reduz** custos com armazenamento colunar
- **Garante** conformidade e segurança

Com as 7 tabelas criadas e o pipeline ETL via Airflow, você está pronto para análises avançadas, machine learning e inteligência de negócios em tempo real.

Guia Completo: dbt para Projeto Apex Motors

Índice

1. Introdução
2. O que é dbt?
3. Arquitetura do Projeto
4. Camadas do dbt
5. Passo a Passo Prático
6. Erros Comuns e Soluções
7. Deployment Environments
8. Próximos Passos

Introdução

Este guia documenta o processo completo de implementação do dbt no projeto Apex Motors, uma concessionária de carros que utiliza engenharia de dados para análise de vendas.

Objetivo: Transformar dados brutos em dados prontos para análise e BI.

O que é dbt?

dbt = **Data Build Tool**

É uma ferramenta que permite:

- ✅ Transformar dados brutos em dados prontos
- ✅ Organizar transformações em camadas
- ✅ Testar qualidade dos dados
- ✅ Documentar o pipeline
- ✅ Versionamento de código

Analogia: Se o Airflow é o “maestro” que coordena, o dbt é o “carpinteiro” que constrói as tabelas!

Arquitetura do Projeto

```
┌──────────────────────────────────────────────────────────────┐
│                PostgreSQL (Render)                           │
│              Dados Brutos da Aplicação                       │
│  (vendas, clientes, veículos, concessionárias, etc)        │
└──────────────────────┬───────────────────────────────────────┘
                       │
                       │
                       │ Airflow (EC2)
                       │ Orquestra o pipeline
                       │
                       ▼
┌──────────────────────────────────────────────────────────────┐
│              BigQuery (Google Cloud)                         │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ┌──────────────────────────────────────────────────┐       │
│  │  RAW (Dados Brutos)                              │       │
│  │  raw_vendas, raw_clientes, raw_veiculos, etc    │       │
│  └──────────────────────────────────────────────────┘       │
│                      │                                       │
│                      │ dbt                                   │
│                      ▼                                       │
│  ┌──────────────────────────────────────────────────┐       │
│  │  STG (Staging - Limpeza)                         │       │
│  │  stg_vendas, stg_clientes, stg_veiculos, etc    │       │
│  └──────────────────────────────────────────────────┘       │
│                      │                                       │
│          ┌───────────┴──────────┐                            │
│          ▼                      ▼                            │
│  ┌────────────────┐      ┌────────────────────┐             │
│  │  DIM           │      │  FACT              │             │
│  │ (Referência)   │      │ (Eventos)          │             │
│  │ dim_clientes   │      │ fct_vendas         │             │
│  │ dim_veiculos   │      │                    │             │
│  │ dim_cidades    │      │ (Conecta DIMs)     │             │
│  └────────────────┘      └────────────────────┘             │
│          │                      │                            │
│          └──────────┬───────────┘                            │
│                     ▼                                        │
│  ┌──────────────────────────────────────────────────┐       │
│  │  ANALYSIS (Análises de Negócio)                  │       │
│  │  analise_vendas_temporal                         │       │
│  │  analise_vendas_concessionaria                   │       │
```

```
│   │  analise_vendas_veiculo                          │   │
│   │  analise_vendas_vendedor                         │   │
│   └──────────────────────────────────────────────────┘   │
│                     │                                    │
│                     ▼                                    │
│   ┌──────────────────────────────────────────────────┐   │
│   │  TEST (Validações)                               │   │
│   │  Valida regras de negócio                        │   │
│   │  Garante qualidade dos dados                     │   │
│   └──────────────────────────────────────────────────┘   │
│                                                          │
└──────────────────────────────────────────────────────────┘
                     │
                     ▼
          ┌──────────────────────┐
          │  BI (Dashboards)     │
          │  Google Data Studio  │
          │  Looker              │
          │  Power BI            │
          └──────────────────────┘
```

Camadas do dbt

1. RAW (Dados Brutos)

O que é: Cópia exata dos dados do PostgreSQL no BigQuery

Características:

- ❌ Nomes inconsistentes
- ❌ Dados sujos
- ✅ Backup dos dados originais
- ✅ Auditoria completa

Exemplo:

```
raw_vendas:
- id_vendas
- id_veiculos
- id_concessionarias
- valor_pago (nome estranho!)
- data_venda
```

---

2. STG (Staging) - Limpeza e Padronização

O que é: Camada intermediária que limpa e padroniza os dados

Funções:

- ✅ Renomear colunas (ex: `valor_pago` → `valor_venda`)
- ✅ Converter tipos de dados
- ✅ Remover dados inválidos
- ✅ Criar IDs únicos
- ✅ Padronizar nomes

Exemplo:

```sql
-- stg_vendas.sql
SELECT
    id_vendas AS venda_id,
    id_veiculos AS veiculo_id,
    valor_pago::DECIMAL(10,2) AS valor_venda,
    data_venda::TIMESTAMP AS data_venda
FROM {{ ref('raw_vendas') }}
WHERE id_vendas IS NOT NULL
```

Resultado:

```
stg_vendas:
- venda_id (renomeado)
- veiculo_id (renomeado)
- valor_venda (renomeado + tipo correto)
- data_venda (tipo correto)
```

---

3. DIM (Dimensions) - Tabelas de Referência

O que é: Tabelas que descrevem "o quê" (entidades, dimensões)

Características:

- ✅ Tabelas pequenas e estáveis
- ✅ Mudança lenta (não mudam frequentemente)
- ✅ Descrevem entidades
- ✅ Usadas para JOIN com FACT

Exemplos:

```sql
-- dim_veiculos.sql
SELECT
    id_veiculos AS veiculo_id,
    nome AS nome_veiculo,
    tipo,
    valor AS valor_sugerido
FROM {{ ref('stg_veiculos') }}

-- dim_clientes.sql
SELECT
    id_clientes AS cliente_id,
    cliente AS nome_cliente,
    endereco
FROM {{ ref('stg_clientes') }}
```

Tabelas DIM no Projeto:

- `dim_veiculos`
- `dim_clientes`

- `dim_vendedores`
- `dim_concessionarias`
- `dim_cidades`
- `dim_estados`

---

4. FACT (Facts) - Tabela Principal de Eventos

O que é: Tabela que registra “o que aconteceu” (eventos, transações)

Características:

- ✅ Tabela GRANDE (muitos registros)
- ✅ Muda frequentemente (novos eventos)
- ✅ Conecta todas as dimensões
- ✅ Armazena métricas

Exemplo:

```sql
-- fct_vendas.sql
{{ config(materialized='table') }}
WITH vendas AS (
    SELECT
        v.id_vendas AS venda_id,
        v.id_veiculos AS veiculo_id,
        v.id_concessionarias AS concessionaria_id,
        v.id_vendedores AS vendedor_id,
        v.id_clientes AS cliente_id,
        v.valor_venda,
        v.data_venda,
        v.data_inclusao,
        v.data_atualizacao
    FROM {{ ref('stg_vendas') }} v
    JOIN {{ ref('dim_veiculos') }} vei ON v.id_veiculos = vei.veiculo_id
    JOIN {{ ref('dim_concessionarias') }} con ON v.id_concessionarias =
con.concessionaria_id
    JOIN {{ ref('dim_vendedores') }} ven ON v.id_vendedores =
ven.vendedor_id
    JOIN {{ ref('dim_clientes') }} cli ON v.id_clientes = cli.cliente_id
)

SELECT
    venda_id,
    veiculo_id,
    concessionaria_id,
    vendedor_id,
    cliente_id,
    valor_venda,
    data_venda,
    data_inclusao,
    data_atualizacao
FROM vendas
```

---

5. ANALYSIS - Análises Específicas

O que é: Tabelas que respondem perguntas específicas do negócio

Características:

- ✅ Dados agregados
- ✅ Pronto para BI

- ✅ Responde perguntas do negócio
- ✅ Pode ter múltiplas análises

Exemplos:

```sql
-- analise_vendas_temporal.sql
SELECT
    DATE_TRUNC(v.data_venda, MONTH) AS mes_venda,
    COUNT(v.venda_id) AS numero_vendas,
    SUM(v.valor_venda) AS total_vendas,
    AVG(v.valor_venda) AS valor_medio_venda
FROM {{ ref('fct_vendas') }} v
GROUP BY DATE_TRUNC(v.data_venda, MONTH)
ORDER BY mes_venda

-- analise_vendas_concessionaria.sql
SELECT
    con.concessionaria_id AS id,
    con.nome_concessionaria AS concessionaria,
    COUNT(v.venda_id) AS quantidade,
    SUM(v.valor_venda) AS total,
    AVG(v.valor_venda) AS valor_medio
FROM {{ ref('fct_vendas') }} v
JOIN {{ ref('dim_concessionarias') }} con ON v.concessionaria_id =
con.concessionaria_id
GROUP BY con.concessionaria_id, con.nome_concessionaria
ORDER BY total DESC
```

---

6. TEST - Validações

O que é: Testes que validam se os dados estão corretos

Características:

- ✅ Garante qualidade dos dados
- ✅ Valida regras de negócio
- ✅ Alertas sobre problemas

Exemplo (Seu Teste):

```sql
-- test.sql
WITH vendas_comparacao_preco AS (
    SELECT
        f.venda_id,
        f.valor_venda,
        d.valor_sugerido,
        CASE
            WHEN f.valor_venda <= d.valor_sugerido
             AND f.valor_venda >= d.valor_sugerido * 0.95
            THEN TRUE
            ELSE FALSE
        END AS regra_respeitada
    FROM {{ ref('fct_vendas') }} f
    JOIN {{ ref('dim_veiculos') }} d ON f.veiculo_id = d.veiculo_id
)

SELECT venda_id, valor_venda, valor_sugerido
FROM vendas_comparacao_preco
WHERE regra_respeitada = FALSE
```

O que Valida:

- ✅ Preço pago está entre 95% e 100% do sugerido
- ✅ Se alguma venda viola, o teste falha
- ✅ Garante qualidade do negócio

Resultado:

- ✅ 0 resultados = Teste passa ✅
- ❌ Qualquer resultado = Teste falha ❌

Passo a Passo Prático

Passo 1: Preparar os Dados no PostgreSQL

Objetivo: Ter dados corretos no PostgreSQL (Render)

Ações:

1. Verificar se os dados estão corretos
2. Criar CSVs com dados corrigidos
3. Importar no PostgreSQL

Resultado: Dados brutos prontos no PostgreSQL

Passo 2: Criar Tabelas Raw no BigQuery

Objetivo: Copiar dados brutos do PostgreSQL para BigQuery

Passo 2.1: Criar as Tabelas

Execute no BigQuery Query Editor:

```sql
CREATE TABLE apex_motors_dw.raw_veiculos (
   id_veiculos INTEGER,
   nome STRING NOT NULL,
   tipo STRING NOT NULL,
   valor DECIMAL(10, 2) NOT NULL,
   data_atualizacao TIMESTAMP,
   data_inclusao TIMESTAMP
);

CREATE TABLE apex_motors_dw.raw_estados (
   id_estados INTEGER,
   estado STRING NOT NULL,
   sigla STRING NOT NULL,
   data_inclusao TIMESTAMP,
   data_atualizacao TIMESTAMP
);

CREATE TABLE apex_motors_dw.raw_cidades (
   id_cidades INTEGER,
   cidade STRING NOT NULL,
   id_estados INTEGER NOT NULL,
   data_inclusao TIMESTAMP,
   data_atualizacao TIMESTAMP
);

CREATE TABLE apex_motors_dw.raw_concessionarias (
   id_concessionarias INTEGER,
   concessionaria STRING NOT NULL,
   id_cidades INTEGER NOT NULL,
   data_inclusao TIMESTAMP,
   data_atualizacao TIMESTAMP
);

CREATE TABLE apex_motors_dw.raw_vendedores (
   id_vendedores INTEGER,
   nome STRING NOT NULL,
   id_concessionarias INTEGER NOT NULL,
   data_inclusao TIMESTAMP,
   data_atualizacao TIMESTAMP
);

CREATE TABLE apex_motors_dw.raw_clientes (
   id_clientes INTEGER,
   cliente STRING NOT NULL,
   endereco STRING NOT NULL,
```

```sql
    id_concessionarias INTEGER NOT NULL,
    data_inclusao TIMESTAMP,
    data_atualizacao TIMESTAMP
);

CREATE TABLE apex_motors_dw.raw_vendas (
    id_vendas INTEGER,
    id_veiculos INTEGER NOT NULL,
    id_concessionarias INTEGER NOT NULL,
    id_vendedores INTEGER NOT NULL,
    id_clientes INTEGER NOT NULL,
    valor_pago DECIMAL(10, 2) NOT NULL,
    data_venda TIMESTAMP,
    data_inclusao TIMESTAMP,
    data_atualizacao TIMESTAMP
);
```

Passo 2.2: Importar os Dados

1. Abra o BigQuery
2. Para cada tabela, clique em “Upload”
3. Selecione o CSV correspondente
4. Configure como CSV com header
5. Clique em “Upload”

Resultado: Tabelas raw preenchidas com dados

Passo 3: Configurar o dbt

Objetivo: Conectar dbt ao BigQuery

Passo 3.1: Criar Conta dbt Cloud

1. Abra: https://cloud.getdbt.com
2. Crie uma conta
3. Conecte ao seu projeto BigQuery

Passo 3.2: Configurar Conexão

1. Em dbt Cloud, vá para “Connections”

2. Clique em “Create Connection”
3. Selecione “BigQuery”
4. Autentique com sua conta Google
5. Selecione o projeto `apex-motors-analytics`

Resultado: dbt conectado ao BigQuery

Passo 4: Criar os Modelos dbt

Objetivo: Criar as transformações (STG, DIM, FACT, ANALYSIS)

Estrutura de Pastas:

```
models/
├── stage/
│   ├── stg_vendas.sql
│   ├── stg_clientes.sql
│   ├── stg_veiculos.sql
│   ├── stg_concessionarias.sql
│   ├── stg_vendedores.sql
│   ├── stg_cidades.sql
│   └── stg_estados.sql
├── dimensions/
│   ├── dim_veiculos.sql
│   ├── dim_clientes.sql
│   ├── dim_vendedores.sql
│   ├── dim_concessionarias.sql
│   ├── dim_cidades.sql
│   └── dim_estados.sql
├── facts/
│   └── fct_vendas.sql
└── analysis/
    ├── analise_vendas_temporal.sql
    ├── analise_vendas_concessionaria.sql
    ├── analise_vendas_veiculo.sql
    └── analise_vendas_vendedor.sql
```

Passo 4.1: Criar Staging Models

Exemplo `stg_vendas.sql`:

```
SELECT
    id_vendas AS venda_id,
    id_veiculos AS veiculo_id,
    id_concessionarias AS concessionaria_id,
    id_vendedores AS vendedor_id,
    id_clientes AS cliente_id,
    valor_pago::DECIMAL(10,2) AS valor_venda,
    data_venda::TIMESTAMP AS data_venda,
    data_inclusao::TIMESTAMP AS data_inclusao,
    data_atualizacao::TIMESTAMP AS data_atualizacao
FROM {{ ref('raw_vendas') }}
WHERE id_vendas IS NOT NULL
```

Passo 4.2: Criar Dimension Models

Exemplo `dim_veiculos.sql`:

```
{{ config(materialized='table') }}

SELECT
    id_veiculos AS veiculo_id,
    nome AS nome_veiculo,
    tipo,
    valor AS valor_sugerido
FROM {{ ref('stg_veiculos') }}
```

Passo 4.3: Criar Fact Model

`fct_vendas.sql`:

```
{{ config(materialized='table') }}
WITH vendas AS (
    SELECT
        v.venda_id,
        v.veiculo_id,
        v.concessionaria_id,
        v.vendedor_id,
        v.cliente_id,
        v.valor_venda,
        v.data_venda,
        v.data_inclusao,
        v.data_atualizacao
    FROM {{ ref('stg_vendas') }} v
    JOIN {{ ref('dim_veiculos') }} vei ON v.veiculo_id = vei.veiculo_id
    JOIN {{ ref('dim_concessionarias') }} con ON v.concessionaria_id =
con.concessionaria_id
    JOIN {{ ref('dim_vendedores') }} ven ON v.vendedor_id = ven.vendedor_id
    JOIN {{ ref('dim_clientes') }} cli ON v.cliente_id = cli.cliente_id
)

SELECT
    venda_id,
    veiculo_id,
    concessionaria_id,
    vendedor_id,
    cliente_id,
    valor_venda,
    data_venda,
    data_inclusao,
    data_atualizacao
FROM vendas
```

Passo 4.4: Criar Analysis Models

Exemplo `analise_vendas_temporal.sql`:

```
{{ config(materialized='table') }}
SELECT
    DATE_TRUNC(v.data_venda, MONTH) AS mes_venda,
    COUNT(v.venda_id) AS numero_vendas,
    SUM(v.valor_venda) AS total_vendas,
    AVG(v.valor_venda) AS valor_medio_venda
FROM {{ ref('fct_vendas') }} v
GROUP BY DATE_TRUNC(v.data_venda, MONTH)
ORDER BY mes_venda
```

Passo 5: Criar Testes

Objetivo: Validar qualidade dos dados

Arquivo: `tests/test.sql`

```
WITH vendas_comparacao_preco AS (
    SELECT
        f.venda_id,
        f.valor_venda,
        d.valor_sugerido,
        CASE
            WHEN f.valor_venda <= d.valor_sugerido
             AND f.valor_venda >= d.valor_sugerido * 0.95
            THEN TRUE
            ELSE FALSE
        END AS regra_respeitada
    FROM {{ ref('fct_vendas') }} f
    JOIN {{ ref('dim_veiculos') }} d ON f.veiculo_id = d.veiculo_id
)

SELECT venda_id, valor_venda, valor_sugerido
FROM vendas_comparacao_preco
WHERE regra_respeitada = FALSE
```

Passo 6: Executar dbt

Objetivo: Criar as tabelas no BigQuery

Comando:

```
dbt run
```

O que acontece:

1. dbt lê os arquivos SQL
2. Cria as tabelas no BigQuery
3. Executa as transformações

Resultado: Tabelas STG, DIM, FACT, ANALYSIS criadas no BigQuery

---

Passo 7: Rodar Testes

Objetivo: Validar qualidade dos dados

Comando:

```
dbt test
```

O que acontece:

1. dbt executa os testes
2. Valida as regras de negócio
3. Retorna 0 se passou, ou número de violações se falhou

Resultado Esperado:

```
✅ 0 resultados = Teste passa!
```

Erros Comuns e Soluções

Erro 1: “Table not found”

Problema:

```
Not found: Table apex-motors-analytics:apex_motors_dw.raw_cidades was not
found
```

Solução:

- Verificar se as tabelas raw foram criadas
- Verificar se os nomes estão corretos
- Verificar se o dataset é `apex_motors_dw`

Erro 2: “Unrecognized name”

Problema:

```
Unrecognized name: cidade_id at [19:5]
```

Solução:

- Verificar o nome correto da coluna
- Executar `SELECT * FROM tabela LIMIT 1` para ver as colunas
- Usar o nome correto no SQL

Erro 3: “Billing has not been enabled”

Problema:

```
Billing has not been enabled for this project
```

Solução:

- Ativar billing no Google Cloud
- Adicionar cartão de crédito
- Vincular ao projeto

Erro 4: "DATE_TRUNC syntax error"

Problema:

```
A valid date part name is required but found v.data_venda
```

Solução:

- BigQuery usa sintaxe diferente: `DATE_TRUNC(data, MONTH)`
- Não use: `DATE_TRUNC('month', data)`

Deployment Environments

O que é?

Separação entre ambiente de desenvolvimento e produção

DEVELOP

- Seu ambiente de trabalho
- Você testa código
- Dados de teste
- Dataset: `apex_motors_dw_dev`

PRODUCTION

- Ambiente real

- Usuários dependem
- Dados reais
- Dataset: `apex_motors_dw`

Como Configurar

Passo 1: Criar Environment DEVELOP

1. Abra dbt Cloud
2. Vá para "Environments"
3. Clique em "Create Environment"
4. Configure:
    - Name: `develop`
    - Type: `Development`
    - Dataset: `apex_motors_dw_dev`

Passo 2: Criar Environment PRODUCTION

1. Clique em "Create Environment"
2. Configure:
    - Name: `production`
    - Type: `Deployment`
    - Dataset: `apex_motors_dw`

Passo 3: Criar Main Job

1. Vá para "Jobs"
2. Clique em "Create Job"
3. Configure:
    - Name: `main_job`
    - Environment: `production`
    - Commands: `dbt run` e `dbt test`
    - Schedule: Diário às 8h

Próximos Passos

1. **Configurar Airflow** para orquestrar o pipeline
2. **Criar Dashboards** no Google Data Studio ou Looker
3. **Monitorar** o pipeline com alertas
4. **Documentar** as transformações
5. **Versionar** o código com Git

Resumo das Camadas

Camada	Função	Exemplo
RAW	Backup dos dados originais	raw_vendas
STG	Limpeza e padronização	stg_vendas
DIM	Tabelas de referência	dim_clientes
FACT	Tabela principal de eventos	fct_vendas
ANALYSIS	Análises de negócio	analise_vendas_temporal
TEST	Validações	test.sql

Conclusão

O dbt é uma ferramenta poderosa que permite:

- ✅ Organizar transformações em camadas
- ✅ Testar qualidade dos dados
- ✅ Documentar o pipeline
- ✅ Versionamento de código
- ✅ Reutilizar transformações

Com dbt, você cria um **Data Warehouse Profissional**! 🚀

Google Looker Studio - Criando Dashboards BI

Introdução ao Google Looker Studio

Google Looker Studio é uma ferramenta de visualização de dados totalmente gratuita oferecida pelo Google Cloud Platform (GCP). Ela permite criar dashboards interativos e relatórios profissionais conectando-se a diversas fontes de dados, como BigQuery, Google Sheets, MySQL, PostgreSQL e muitas outras.

Looker Studio é a escolha ideal para empresas que precisam de análise de dados em tempo real, inteligência de negócios (BI) e criação de relatórios executivos sem necessidade de infraestrutura complexa ou custos elevados de licenciamento.

Por que Usar Looker Studio

Característica	Descrição
Totalmente Gratuito	Sem custos de licença ou infraestrutura
Sem Configuração	Conecta-se diretamente a BigQuery e outras fontes
Dashboards Interativos	Filtros, drill-downs e exploração de dados em tempo real
Integração com Google Cloud	Funciona perfeitamente com BigQuery, Google Sheets, Analytics
Compartilhamento Fácil	Compartilhe dashboards com links ou permissões granulares
Múltiplas Visualizações	Gráficos de barras, pizza, linhas, tabelas, mapas e muito mais
Filtros e Parâmetros	Crie filtros interativos para exploração de dados
Responsivo	Funciona em desktop, tablet e mobile
Colaboração em Tempo Real	Múltiplos usuários podem editar simultaneamente

---

Principais Características e Benefícios

Looker Studio oferece um conjunto completo de recursos para criar dashboards profissionais:

Visualizações Ricas: Escolha entre mais de 20 tipos de gráficos diferentes, incluindo gráficos de barras, linhas, pizza, mapas geográficos, tabelas, scorecards e muito mais.

Filtros Interativos: Adicione filtros dinâmicos que permitem aos usuários explorar dados por período, região, produto ou qualquer outra dimensão relevante. Os filtros são aplicados em tempo real a todos os gráficos.

Combinação de Dados: Combine dados de múltiplas fontes (BigQuery, Sheets, MySQL, etc.) em um único dashboard usando relacionamentos e joins.

Controles Personalizados: Adicione controles como seletores de data, dropdowns e campos de texto para criar dashboards altamente interativos.

Temas e Formatação: Personalize cores, fontes, layouts e crie temas corporativos que refletem a identidade visual da sua empresa.

Compartilhamento e Permissões: Controle quem pode visualizar, editar ou compartilhar cada dashboard com permissões granulares.

Conceitos Fundamentais

Fonte de Dados

Uma **fonte de dados** é a conexão entre Looker Studio e um banco de dados ou planilha. Você pode conectar-se a BigQuery, Google Sheets, MySQL, PostgreSQL, Salesforce, Google Analytics e muitas outras plataformas.

Para o projeto Apex Motors, usaremos **BigQuery** como fonte de dados, conectando-nos ao dataset `apex_motors_dw` que contém as tabelas transformadas (fct_vendas, dim_veiculos, dim_concessionarias, etc).

Dimensões e Métricas

Dimensões são atributos categóricos (texto) que agrupam dados, como data, nome de concessionária, modelo de veículo ou vendedor. Elas aparecem geralmente no eixo X de gráficos.

Métricas são valores numéricos que podem ser agregados, como soma de vendas, contagem de registros, média de preço ou quantidade de unidades. Elas aparecem geralmente no eixo Y de gráficos.

Gráficos

Looker Studio oferece diversos tipos de gráficos para diferentes necessidades:

Tipo de Gráfico	Melhor Para	Exemplo
Gráfico de Linhas	Tendências ao longo do tempo	Vendas por mês
Gráfico de Barras	Comparação entre categorias	Top 5 concessionárias
Gráfico de Pizza	Proporções e percentuais	% de modelos vendidos
Scorecard	Métricas principais (KPIs)	Total de vendas
Tabela	Dados detalhados	Lista de vendedores
Mapa Geográfico	Distribuição por localização	Vendas por estado

Filtros

Filtros permitem que os usuários do dashboard explorem dados de forma interativa. Um filtro pode ser aplicado a um gráfico específico ou a todo o dashboard. Exemplos: filtro por data, concessionária, vendedor ou modelo de veículo.

Arquitetura do Dashboard Apex Motors

O dashboard de vendas da Apex Motors segue a seguinte estrutura:

```
Dashboard Executivo
├── Título: "Dashboard de Vendas - Apex Motors"
├── Scorecards (KPIs)
│   ├── Total de Vendas (R$)
│   ├── Quantidade de Vendas
│   └── Total de Concessionárias
├── Gráfico de Linhas
│   └── Vendas por Mês (2023-2025)
├── Gráfico de Pizza
│   └── % de Modelos Vendidos
├── Gráfico de Barras
│   └── Vendas por Ano (2023, 2024, 2025)
└── Tabela
    └── Top 10 Concessionárias por Quantidade
```

Passo a Passo: Criar Dashboard de Vendas

Passo 1: Acessar Looker Studio

1. Abra o navegador e acesse https://looker.google.com/
2. Faça login com sua conta Google
3. Clique em **"Criar"** e selecione **"Relatório em branco"**
4. Nomeie o relatório como **"Dashboard de Vendas - Apex Motors"**

Passo 2: Conectar ao BigQuery

1. Na página do novo relatório, clique em **"Adicionar dados"**
2. Procure por **"BigQuery"** e clique nele
3. Selecione seu **projeto GCP** (ex: apex-motors-analytics)
4. Selecione o **dataset** `apex_motors_dw`
5. Selecione a **tabela** `fct_vendas`
6. Clique em **"Conectar"**

Passo 3: Adicionar Título

1. Clique em **"Inserir"** → **"Caixa de texto"**
2. Digite: **"Dashboard de Vendas - Apex Motors"**
3. Formate:
   - Tamanho da fonte: **24-28**
   - Negrito: **SIM**
   - Cor: **Azul ou Preto**
4. Posicione no topo do dashboard

Passo 4: Adicionar Scorecard - Total de Vendas

1. Clique em **"Adicionar um gráfico"** → **"Scorecard"**
2. Na aba **"Dados"**:

- **Dimensão**: Deixe em branco
- **Métrica**: `SUM(valor_venda)`

3. Na aba **"Estilo"**:
   - **Título**: "Total de Vendas"
   - **Formato**: Moeda (R$)
4. Posicione à esquerda do dashboard

Passo 5: Adicionar Scorecard - Quantidade de Vendas

1. Clique em **"Adicionar um gráfico"** → **"Scorecard"**
2. Na aba **"Dados"**:
   - **Dimensão**: Deixe em branco
   - **Métrica**: `COUNT(venda_id)`
3. Na aba **"Estilo"**:
   - **Título**: "Total de Vendas"
   - **Formato**: Número inteiro
4. Posicione no centro

Passo 6: Adicionar Scorecard - Total de Concessionárias

1. Clique em **"Adicionar um gráfico"** → **"Scorecard"**
2. Na aba **"Dados"**:
   - **Dimensão**: Deixe em branco
   - **Métrica**: `COUNT(DISTINCT concessionaria_id)`
3. Na aba **"Estilo"**:
   - **Título**: "Total de Concessionárias"
   - **Formato**: Número inteiro
4. Posicione à direita

Passo 7: Adicionar Gráfico de Linhas - Vendas por Mês

1. Clique em **"Adicionar um gráfico"** → **"Gráfico de linhas"**

2. Na aba **“Dados”**:
    - **Dimensão (eixo X)**: `data_venda` (Data)
    - **Métrica (eixo Y)**: `SUM(valor_venda)`
3. Na aba **“Estilo”**:
    - **Título**: “Vendas ao longo do tempo”
    - **Nível de detalhamento**: Mês
4. Posicione no centro do dashboard

Passo 8: Adicionar Gráfico de Pizza - % Modelos Vendidos

1. Clique em **“Adicionar um gráfico”** → **“Gráfico de pizza”**
2. Na aba **“Dados”**:
    - **Combine dados** com a tabela `dim_veiculos`
    - **Dimensão**: `modelo` (de dim_veiculos)
    - **Métrica**: `COUNT(venda_id)`
3. Na aba **“Estilo”**:
    - **Título**: “% Modelos Vendidos”
4. Posicione à esquerda

Passo 9: Adicionar Tabela - Top 10 Concessionárias

1. Clique em **“Adicionar um gráfico”** → **“Tabela”**
2. Na aba **“Dados”**:
    - **Combine dados** com a tabela `dim_concessionarias`
    - **Dimensão**: `nome` (de dim_concessionarias)
    - **Métrica**: `COUNT(venda_id)`
3. Na aba **“Estilo”**:
    - **Título**: “Top 10 Concessionárias”
    - **Limite de linhas**: 10
4. Posicione à direita

Passo 10: Adicionar Filtros Interativos

1. Clique em **"Adicionar um controle"** → **"Filtro de data"**
2. Configure:
   - **Campo**: `data_venda`
   - **Tipo**: Intervalo de datas
3. Clique em **"Adicionar um controle"** → **"Filtro de seleção"**
4. Configure:
   - **Campo**: `concessionaria_id`
   - **Tipo**: Dropdown

Passo 11: Salvar e Compartilhar

1. Clique em **"Salvar"** no canto superior direito
2. Para compartilhar:
   - Clique em **"Compartilhar"**
   - Adicione emails ou crie um link público
   - Defina permissões (visualizar ou editar)

Consultas SQL Úteis para Looker Studio

Top 5 Concessionárias por Vendas

```sql
SELECT
  c.nome as concessionaria,
  COUNT(*) as quantidade_vendas,
  SUM(f.valor_venda) as valor_total
FROM `apex-motors-analytics.apex_motors_dw.fct_vendas` f
JOIN `apex-motors-analytics.apex_motors_dw.dim_concessionarias` c
  ON f.concessionaria_id = c.concessionaria_id
GROUP BY c.nome
ORDER BY quantidade_vendas DESC
LIMIT 5
```

Modelos Mais Vendidos

```sql
SELECT
  v.modelo,
  COUNT(*) as quantidade_vendas,
  SUM(f.valor_venda) as valor_total
FROM `apex-motors-analytics.apex_motors_dw.fct_vendas` f
JOIN `apex-motors-analytics.apex_motors_dw.dim_veiculos` v
  ON f.veiculo_id = v.veiculo_id
GROUP BY v.modelo
ORDER BY quantidade_vendas DESC
```

Vendas por Mês

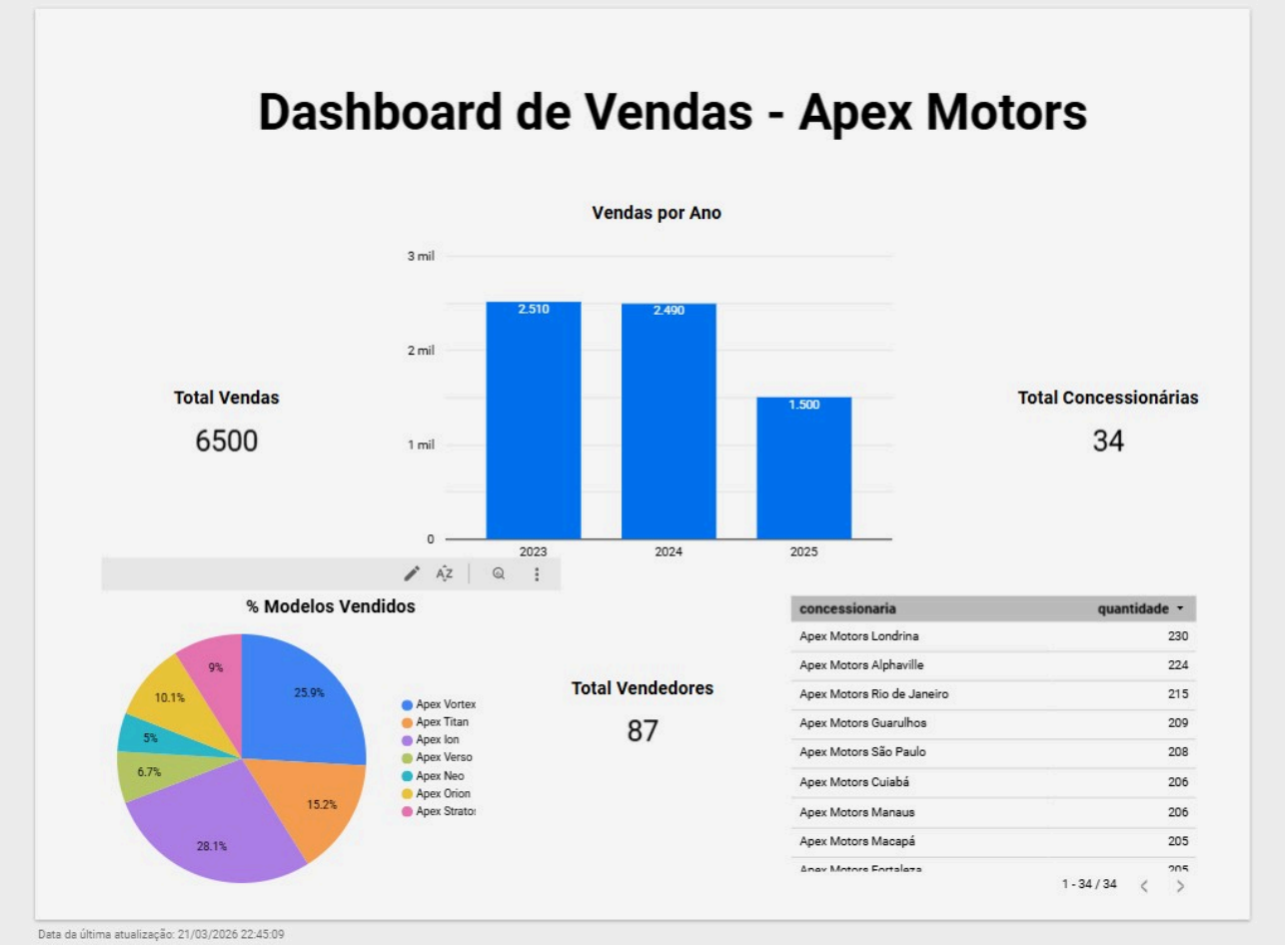
```sql
SELECT
  DATE_TRUNC(f.data_venda, MONTH) as mes,
  COUNT(*) as quantidade,
  SUM(f.valor_venda) as valor_total
FROM `apex-motors-analytics.apex_motors_dw.fct_vendas` f
GROUP BY mes
ORDER BY mes DESC
```

Top 5 Vendedores

```sql
SELECT
  v.nome as vendedor,
  COUNT(*) as quantidade_vendas,
  SUM(f.valor_venda) as valor_total
FROM `apex-motors-analytics.apex_motors_dw.fct_vendas` f
JOIN `apex-motors-analytics.apex_motors_dw.dim_vendedores` v
  ON f.vendedor_id = v.vendedor_id
GROUP BY v.nome
ORDER BY quantidade_vendas DESC
LIMIT 5
```

Dashboard Apex Motors - Resultado Final

O dashboard de vendas da Apex Motors foi implementado com sucesso, consolidando dados de 6.500 vendas em um painel executivo interativo. A imagem abaixo mostra o resultado final:



Componentes Visíveis:

- **Scorecards**: Total de Vendas (6.500), Total de Vendedores (87), Total de Concessionárias (34)
- **Gráfico de Barras**: Vendas por Ano (2023: 2.510, 2024: 2.490, 2025: 1.500)
- **Gráfico de Pizza**: % Modelos Vendidos (Apex Vortex lidera com 25.9%)
- **Tabela**: Top 10 Concessionárias por Quantidade de Vendas
- **Data de Atualização**: 21/03/2026 22:45:09

Boas Práticas para Dashboards BI

Mantenha a Simplicidade: Um dashboard deve comunicar informações-chave rapidamente. Evite adicionar muitos gráficos ou métricas desnecessárias que possam confundir o usuário.

Use Cores Consistentes: Defina uma paleta de cores corporativa e mantenha consistência em todo o dashboard. Cores devem ter significado (ex: vermelho para alertas, verde para sucesso).

Agrupe Informações Relacionadas: Organize gráficos de forma lógica, agrupando métricas relacionadas próximas uma da outra.

Adicione Contexto: Use títulos descritivos, legendas e notas explicativas para que os usuários entendam o que estão vendo.

Otimize para Performance: Evite consultas muito complexas ou que retornem grandes volumes de dados. Use filtros para reduzir o escopo dos dados.

Teste com Usuários Finais: Compartilhe o dashboard com usuários finais e colete feedback para melhorias contínuas.

Mantenha Atualizado: Revise regularmente os dados e métricas para garantir que o dashboard continua relevante e preciso.

Conclusão

Google Looker Studio é uma ferramenta poderosa e acessível para criar dashboards profissionais de BI. Seguindo este guia, você conseguiu criar um dashboard executivo completo para a Apex Motors que mostra vendas, modelos mais vendidos e desempenho das concessionárias.

Com prática, você poderá criar dashboards cada vez mais sofisticados e personalizados para diferentes públicos e necessidades de negócio.